ESP32-S3 HAL — Reference Manual

Generated from the Ada package specifications

Contents

This reference is generated from the package specifications under `libs/esp32s3_hal/`. For each driver it shows the package's header documentation and its public API (types and operations). The generated register layer (`svd/`) is summarised at the end.

Peripheral & device drivers (`src/`)

ESP32S3

Source: `src/esp32s3.ads`

Root of the reusable ESP32-S3 peripheral drivers (the hand-written HAL).

The thin register layer lives under ESP32S3_Registers.* (svd2ada-generated; see ../svd and ../regenerate.sh) and is never hand-edited. Driver packages are children of this one: ESP32S3.GPIO, ESP32S3.RNG, ESP32S3.Temperature, ESP32S3.GDMA, ESP32S3.SPI (and, later, ESP32S3.UART, ESP32S3.I2S, ...). Consumers `with "esp32s3_hal.gpr";` (by name, or by relative path in-repo) then `with ESP32S3.<Peripheral>;`. See ../README.md.

Public API

```
   pragma Pure;
```

ESP32S3.ADC

Source: `src/esp32s3-adc.ads`

ESP32-S3 SAR ADC (analog-to-digital converter) – one-shot reads.

```
Two ADC units, each with up to ten 12-bit channels mapped to fixed GPIOs:
  ADC1 channel n -> GPIO (n + 1)   (ch0 = GPIO1 .. ch9 = GPIO10)
  ADC2 channel n -> GPIO (n + 11)  (ch0 = GPIO11 .. ch9 = GPIO20)
This driver does software-triggered single conversions through the RTC
controller.  Each conversion returns a raw 0 .. 4095 code; per-channel input
attenuation sets the full-scale voltage (0 dB ~ 1.1 V .. 12 dB ~ 3.3 V).
```

Ownership: a unit is a shared resource handed out as a CLAIMED handle, LIMITED (non-copyable) and CONTROLLED (released automatically on scope exit). Uses finalization, so it targets the embedded/full profile.

Public API

```
   type ADC_Unit is (ADC1, ADC2);
   type Channel_Index is range 0 .. 9;

   --  Input attenuation -> approximate full-scale input voltage.
   type Attenuation is (Db_0, Db_2_5, Db_6, Db_12);

   subtype Raw_Value is Natural range 0 .. 4095;     --  12-bit conversion result
```

```
   --  Non-copyable handle to a claimed ADC unit (check Is_Valid after Claim).
   type Reader is limited private;

   procedure Claim (R : in out Reader; Unit : ADC_Unit);
   function Is_Valid (R : Reader) return Boolean;
   procedure Release (R : in out Reader);

   --  Read one sample from channel Ch at the given attenuation (blocking; a
   --  conversion takes a few microseconds).  Returns 0 on an invalid handle.
   function Read (R : Reader; Ch : Channel_Index;
                  Atten : Attenuation := Db_12) return Raw_Value;

   --  The GPIO a unit's channel is wired to (for routing / documentation).
   function Channel_Pin (Unit : ADC_Unit; Ch : Channel_Index)
                         return ESP32S3.GPIO.Pin_Id;

   --  Diagnostics: the self-calibrated initial code, and whether the most recent
   --  conversion's DONE flag asserted (False => the SAR did not convert).
   function Cal_Code (Unit : ADC_Unit) return Natural;
   function Last_Done return Boolean;
```

ESP32S3.AES

Source: `src/esp32s3-aes.ads`

ESP32-S3 AES accelerator – single-block AES ECB (the building block).

The accelerator is a single shared resource, so a protected object serialises the key/text load, trigger and read, making concurrent calls from different tasks safe. This exposes one-block ECB encrypt and decrypt for 128- and 256-bit keys (the hardware also does chaining modes). Register pokes, no finalization – works under every runtime profile.

NOTE: the ESP32-S3 AES hardware supports only 128- and 256-bit keys – there is no 192-bit support on this silicon (selecting "AES-192" makes the engine silently fall back to AES-128 on the first 16 key bytes). 192-bit keys exist only on the original ESP32. The Supported_Key precondition below makes any unsupported key length a contract violation instead of a silent fallback.

Public API

```
   type Block is array (0 .. 15) of Interfaces.Unsigned_8;   --  128-bit block

   --  Keys of the two hardware-supported sizes (packed big to little internally).
   type Key_Bytes is array (Natural range <>) of Interfaces.Unsigned_8;
   subtype Key_128 is Key_Bytes (0 .. 15);   --  128-bit key
   subtype Key_256 is Key_Bytes (0 .. 31);   --  256-bit key

   --  The only key lengths the S3 AES engine actually implements (in bytes).
   --  Use this in the operation contracts so a wrong-sized key (e.g. a 24-byte
   --  "AES-192" key, which the hardware cannot do) is rejected rather than
   --  silently degraded.  Callers that pass the Key_128 / Key_256 subtypes
   --  satisfy it statically, so there is no run-time cost for correct code.
   function Supported_Key (Key : Key_Bytes) return Boolean is
     (Key'Length = 16 or else Key'Length = 32);
```

```
   --  Key length selects the cipher: 16 bytes => AES-128, 32 => AES-256 (use the
   --  Key_128 / Key_256 subtypes).
   function Encrypt_ECB (Key : Key_Bytes; Plain  : Block) return Block
     with Pre => Supported_Key (Key);
   function Decrypt_ECB (Key : Key_Bytes; Cipher : Block) return Block
     with Pre => Supported_Key (Key);
```

ESP32S3.CH422G

Source: `src/esp32s3-ch422g.ads`

WCH CH422G I2C I/O expander: 8 bidirectional pins (IO0..IO7) + 4 output-only pins (OC0..OC3). Same two-level-locking shape as ESP32S3.TCA9555 – acquire the device like the RTC, while the I2C host is locked only for each transaction – but the chip itself is quite different:

```
* It is NOT a register-pointer device.  Each operation is a single-byte
  transaction to a FIXED, function-specific I2C address: 0x24 set system
  config (WR-SET), 0x23 write OC outputs (WR-OC), 0x38 write IO outputs
  (WR-IO), 0x26 read IO inputs (RD-IO).  So there is one CH422G per bus and
  no address straps -- Setup takes no address.
* IO direction is GLOBAL: one IO_OE bit makes ALL of IO0..IO7 inputs or ALL
  outputs (no per-pin direction).  OC0..OC3 are output-only, globally
  push-pull or open-drain (OD_EN).
* The config / OC / IO-output registers cannot be read back (only the IO
  pins read back, via RD-IO), so the driver keeps a shadow of them,
  initialised to the datasheet power-on defaults (IO = inputs, OC = high,
  push-pull).
* The chip has NO interrupt output -> no .Interrupts child.
```

Uses a controlled Session (finalization) => embedded / full profiles only.

Public API

```
   --  IO0..IO7 as one byte (bit i = IOi); OC0..OC3 as the low nibble.
   type IO_Value is mod 2 ** 8;
   type OC_Value is mod 2 ** 4;

   type IO_Pin is range 0 .. 7;
   type OC_Pin is range 0 .. 3;

   type Pin_State is (Low, High);

   --  GLOBAL settings (the chip has no per-pin direction / drive).
   type IO_Direction is (Inputs, Outputs);      --  all of IO0..IO7
   type OC_Drive     is (Push_Pull, Open_Drain);  --  all of OC0..OC3

   --  Bus_Error: the chip did not ACK (absent / wrong bus / stuck).
   type Status is (OK, Bus_Error);

   type Device  is limited private;
   type Session is limited private;

   --  Raised by Acquire on an un-Setup Device; by an operation whose Session
   --  does not currently hold the device.
```

```
   Not_Initialized : exception;
   Not_Owned       : exception;

   ---------------------------------------------------------------------------
   --  One-time configuration -- call once at startup (single-threaded).  Records
   --  the wiring and brings the I2C host up; does NOT touch the chip (it powers
   --  up as IO-inputs / OC-high / I/O-expansion).  No address -- the CH422G uses
   --  fixed command addresses.
   ---------------------------------------------------------------------------
   procedure Setup
     (Dev      : out Device;
      Sda      : ESP32S3.GPIO.Pin_Id;
      Scl      : ESP32S3.GPIO.Pin_Id;
      Host     : ESP32S3.I2C.I2C_Host := ESP32S3.I2C.I2C0;
      Clock_Hz : Positive             := 400_000);

   ---------------------------------------------------------------------------
   --  Take / release exclusive ownership of the device.
   ---------------------------------------------------------------------------
   procedure Acquire (S : in out Session; Dev : Device);
   procedure Release (S : in out Session);

   --  Address-only probe of the config command address: True iff the chip ACKs.
   function Present (S : Session) return Boolean;

   ---------------------------------------------------------------------------
   --  Operations -- each takes the held Session and locks the I2C host only for
   --  its own transaction.  Raise Not_Owned unless S currently holds the device.
   ---------------------------------------------------------------------------

   --  System config (WR-SET): IO direction (all 8) + OC drive (all 4).  Keeps
   --  the current sleep state; holds A_SCAN = 0 (I/O-expansion mode).  Does not
   --  change any pin's output value.
   procedure Configure
     (S       : Session;
      IO_Dir  : IO_Direction := Inputs;
      OC_Mode : OC_Drive     := Push_Pull;
      Result  : out Status);

   --  Low-power sleep (woken by an IO level change or the next command).
   procedure Sleep (S : Session; On : Boolean; Result : out Status);

   --  IO0..IO7 outputs (WR-IO) -- effective only after Configure (Outputs).
   procedure Write_IO (S : Session; Value : IO_Value; Result : out Status);
   procedure Write_IO_Pin
     (S : Session; Pin : IO_Pin; State : Pin_State; Result : out Status);

   --  IO0..IO7 current pin state (RD-IO).
   procedure Read_IO (S : Session; Value : out IO_Value; Result : out Status);
   procedure Read_IO_Pin
     (S : Session; Pin : IO_Pin; State : out Pin_State; Result : out Status);

   --  OC0..OC3 outputs (WR-OC).  1 = high (push-pull) / not-driven (open-drain),
```

```
   --  0 = low.
   procedure Write_OC (S : Session; Value : OC_Value; Result : out Status);
   procedure Write_OC_Pin
     (S : Session; Pin : OC_Pin; State : Pin_State; Result : out Status);
```

ESP32S3.ES8311

Source: src/esp32s3-es8311.ads

ESP32-S3 driver for the Everest ES8311 low-power mono audio codec.

The ES8311 is controlled over I2C and carries audio over I2S. This driver brings it up for AUDIO OUTPUT (the mono DAC): it runs the I2C register-init sequence and configures an I2S port as the master that generates MCLK, BCLK (SCLK) and LRCK and shifts 16-bit PCM out on the codec's DSDIN line. The codec runs as an I2S slave clocked from the ESP's MCLK = 256 x sample-rate (4.096 MHz at 16 kHz); the register coefficients are the codec-vendor's for that 256x / 16-bit configuration (rate-independent).

Concurrent access is guarded the same way as the other drivers: audio output goes through a limited, non-copyable, CONTROLLED Output handle that owns the I2S port exclusively and releases it automatically on scope exit. Requires a tasking runtime (embedded/full profile), like ESP32S3.I2S.

Register sequence + clock coefficients ported from Espressif's es8311 driver (esp-bsp); see the example README for the source links.

Public API

```
   --  I2C 7-bit address: 0x18 with CE/ADO low (default), 0x19 with CE high.
   subtype Address is ESP32S3.I2C.Slave_Address;
   Default_Address : constant Address := 16#18#;

   ------------------------------------------------------------------------
   --  One-time bring-up -- call once at startup, single-threaded, before any
   --  task contends for the audio port.
   ------------------------------------------------------------------------

   --  Initialise the codec for 16-bit output at Sample_Rate (the I2S then runs
   --  MCLK = 256 x Sample_Rate).  Sda/Scl are the I2C control pins (the bus is
   --  brought up here); Mclk/Sclk/Lrck/Dsdin are the I2S pads.  The codec's
   --  ASDOUT (its ADC out) is not used for output and is left unwired.
   --  Ok is False if the codec did not ACK on I2C (check wiring/address).
   procedure Setup
     (I2C_Bus      : ESP32S3.I2C.I2C_Host;
      Sda          : ESP32S3.GPIO.Pin_Id;
      Scl          : ESP32S3.GPIO.Pin_Id;
      Port         : ESP32S3.I2S.I2S_Port;
      Mclk         : ESP32S3.GPIO.Pin_Id;
      Sclk         : ESP32S3.GPIO.Pin_Id;
      Lrck         : ESP32S3.GPIO.Pin_Id;
      Dsdin        : ESP32S3.GPIO.Pin_Id;
      Sample_Rate  : Positive := 16_000;
      Volume       : Natural  := 70;          --  DAC volume, 0 .. 100 %
      I2C_Clock_Hz : Positive := 100_000;
      Addr         : Address  := Default_Address;
      Ok           : out Boolean);
```

```
   --  Set the DAC output volume (0 .. 100 %).  Setup must have run.
   procedure Set_Volume (Percent : Natural; Ok : out Boolean);

   ------------------------------------------------------------------------
   --  Concurrent, mutually-exclusive audio output.
   ------------------------------------------------------------------------

   --  Raised by Acquire if Setup was never run.
   Not_Ready : exception;

   --  An exclusive hold on the codec's audio (I2S) port.  Limited (cannot be
   --  copied) and CONTROLLED (auto-releases on scope exit, incl. on exception).
   type Output is limited private;

   --  Take exclusive ownership of the audio port (suspends until it is free).
   procedure Acquire (O : in out Output);

   --  Shift Length BYTES of 16-bit PCM (interleaved L/R frames; the mono codec
   --  plays the left slot) out to the codec.  Blocking; Length 1 .. 4095.  Note
   --  back-to-back Play calls leave a brief gap between buffers (an audible
   --  click for a continuous tone); use Play_Continuous for gapless playback.
   procedure Play (O : Output; Samples : System.Address; Length : Natural);

   --  Start playing the Samples buffer on a self-looping DMA and return
   --  immediately: it is replayed forever with NO gap between passes (gapless,
   --  click-free) and zero CPU cost after the call.  Samples must stay valid
   --  (in internal SRAM) and should hold a whole number of wave periods so the
   --  wrap is seamless -- e.g. for a steady tone, size the buffer to an integer
   --  number of cycles.  Length 1 .. 4095 bytes.  Stop halts it.
   procedure Play_Continuous (O : Output; Samples : System.Address;
                              Length : Natural);

   --  Stop a continuous playback started by Play_Continuous.
   procedure Stop (O : Output);

   --  Relinquish the port (also done automatically on scope exit).
   procedure Release (O : in out Output);
```

ESP32S3.Fonts

Source: *src/esp32s3-fonts.ads*

Panel-independent text/font data model for proportional bitmap fonts.

A Font is a light descriptor that POINTS (by address) at flat glyph-atlas arrays generated offline (see libs/esp32s3_hal/tools/gen_font.py): per-glyph metrics (advance, size, bearings, byte offset) plus packed coverage. Two coverage encodings are supported:

```
Bpp = 4  anti-aliased: 4-bit (16-level) coverage, 2 px/byte.
Bpp = 1  monochrome:   1-bit coverage, 8 px/byte (MSB first), ~4x smaller.
```

This package has NO display dependency – it only models the glyph data and reads it through accessor functions. The actual rasterising/blitting is done by the generic ESP32S3.Fonts.Render, instantiated per panel (e.g. ESP32S3.ST7789.Fonts). The atlas data and Font values are therefore reusable across panels

unchanged.

Public API

```
   --  Atlas array element types (the generated atlas packages import these).
   type Byte_Array  is array (Natural range <>) of Interfaces.Unsigned_8;
   type SByte_Array is array (Natural range <>) of Interfaces.Integer_8;
   type U16_Array   is array (Natural range <>) of Interfaces.Unsigned_16;

   --  An 8-bit-per-channel colour the renderer blends in; the panel instance
   --  maps it to its own pixel format via its To_Color formal.
   type RGB is record
      R, G, B : Natural := 0;    --  each 0 .. 255
```

ESP32S3.Fonts.Render

Source: *src/esp32s3-fonts-render.ads*

Public API

```
   --  Draw one character with its baseline left corner at (X, Baseline).
   procedure Draw_Char
     (S        : Surface;
      F        : ESP32S3.Fonts.Font;
      X        : Integer;
      Baseline : Integer;
      Ch       : Character;
      FG, BG   : ESP32S3.Fonts.RGB);

   --  Draw Str with its baseline left end at (X, Baseline); FG over a known BG
   --  (the caller has painted BG behind the text run).  Codes not in F are
   --  skipped.  Advances proportionally by each glyph's metric.
   procedure Draw_Text
     (S        : Surface;
      F        : ESP32S3.Fonts.Font;
      X        : Integer;
      Baseline : Integer;
      Str      : String;
      FG, BG   : ESP32S3.Fonts.RGB);
```

ESP32S3.GDMA

Source: *src/esp32s3-gdma.ads*

ESP32-S3 General DMA (GDMA).

The S3 has ONE AHB GDMA block with five channel pairs (0 .. 4). Each pair has an independent transmit (OUT) path and receive (IN) path, and either can be wired to any peripheral through the GDMA crossbar – the channel is a runtime-assigned resource, not a fixed-per-peripheral one. This driver models that directly: Claim hands out a free Channel handle bound to a peripheral; later peripheral drivers (SPI, I2S, ...) will Claim a channel and drive their own descriptors over it.

Transfers are described by linked lists of descriptors in memory; the engine walks the list, moving each

buffer. This first cut implements the memory-to-memory path (Mem2Mem): a single Copy that the hardware completes by looping one channel's OUT path into its own IN path.

Concurrency / ownership: Claim / Release go through a protected allocator, so concurrent tasks can never be handed the same channel. The Channel handle is LIMITED (non-copyable – you can't assign it to another variable or pass a copy to another task, so two tasks can't alias one channel) and CONTROLLED (it releases its channel automatically when it leaves scope, including on an exception, so a channel can't leak or be reused through a stale copy). Once you hold a Channel, only you touch its registers and descriptors – so the transfer operations need no further locking. Using finalization, this driver targets the embedded/full profile (excluded from the light-tasking build).

Public API

```
   --  The five GDMA channel pairs.
   type Channel_Id is mod 5;

   --  Peripherals a channel can be bound to.  Mem2Mem is the internal
   --  memory-to-memory loopback (no external peripheral).  The others match the
   --  GDMA PERI_SEL encoding and are placeholders until their drivers land.
   type Peripheral is
     (Mem2Mem, SPI2, SPI3, UHCI0, I2S0, I2S1, LCD_CAM, AES, SHA, ADC_DAC, RMT);

   --  An opaque, non-copyable handle to a claimed channel.  Default-initialised
   --  to invalid (check Is_Valid); auto-releases its channel on scope exit.
   type Channel is limited private;

   --  Largest single-descriptor transfer (hardware buffer-size field is 12 bits).
   Max_Transfer : constant := 4095;

   --  Claim a free channel into C and bind both its paths to Peri.  If all five
   --  are in use, C is left invalid (Is_Valid False).  For Mem2Mem the one
   --  channel's OUT and IN paths are used together.  (If C already holds a
   --  channel it is released first.)  C releases its channel automatically when
   --  it goes out of scope -- call Release only to hand it back early.
   procedure Claim (C : in out Channel; Peri : Peripheral);

   --  True when Claim succeeded (a real channel is held).
   function Is_Valid (C : Channel) return Boolean;

   --  Return a channel to the free pool (also tears down its peripheral
   --  binding).  Harmless on an invalid handle.
   procedure Release (C : in out Channel);

   --  Blocking memory-to-memory copy of Length bytes (1 .. Max_Transfer) from
   --  Src to Dst over channel C (claimed for Mem2Mem).  Returns once the
   --  transfer's success-EOF is observed.  Src, Dst and the driver's internal
   --  descriptors must live in INTERNAL SRAM -- the descriptor link address is a
   --  20-bit field the engine completes within the on-chip RAM region.
   --
   --  No-op if C is invalid or Length is 0 / over Max_Transfer.
   procedure Copy (C : Channel; Dst, Src : System.Address; Length : Natural);

   ----------------------------------------------------------------------
   --  Peripheral-bound transfers.
```

```
   --
   --  A peripheral driver (SPI, I2S, UART/UHCI, ...) Claims a channel bound to
   --  its peripheral, then drives one direction at a time over it.  The GDMA
   --  side here is the same descriptor engine the (HW-verified) Copy uses; the
   --  peripheral's own configuration and its DMA request handshake live in the
   --  peripheral driver.
   ------------------------------------------------------------------------

   --  Data-flow direction of a transfer:
   --    Mem_To_Periph -> the OUT (TX) path reads RAM and feeds the peripheral
   --    Periph_To_Mem -> the IN (RX) path receives from the peripheral into RAM
   type Direction is (Mem_To_Periph, Periph_To_Mem);

   --  Arm a single-buffer transfer in direction Dir on channel C and kick the
   --  GDMA side.  NON-blocking: configure and start the peripheral separately;
   --  the GDMA moves data as the peripheral asserts its DMA request.  Poll Done
   --  or call Wait for completion.  Buffer must be in internal SRAM; Length in
   --  1 .. Max_Transfer.  No-op on an invalid handle or out-of-range Length.
   procedure Start (C : Channel; Dir : Direction;
                    Buffer : System.Address; Length : Natural);

   --  Arm a CONTINUOUS (self-looping) transmit on C's OUT path: a single
   --  descriptor whose link points back to itself, so the engine replays
   --  Buffer forever with no gap between passes.  NON-blocking and never
   --  completes on its own -- the peripheral keeps consuming Buffer until the
   --  channel is Released (or the peripheral is stopped).  For gapless looped
   --  playback of a periodic waveform (e.g. a steady tone) with zero CPU
   --  involvement after the kick.  Buffer must be in internal SRAM and should
   --  hold a whole number of wave periods so the wrap is seamless; Length in
   --  1 .. Max_Transfer.  No-op on an invalid handle or out-of-range Length.
   procedure Start_Loop (C : Channel; Buffer : System.Address; Length : Natural);

   --  True once the Dir transfer has signalled success-EOF (also True for an
   --  invalid handle, so a Wait never hangs on one).
   function Done (C : Channel; Dir : Direction) return Boolean;

   --  Block (bounded busy-wait) until Done (C, Dir).
   procedure Wait (C : Channel; Dir : Direction);
```

ESP32S3.GPIO

Source: `src/esp32s3-gpio.ads`

ESP32-S3 GPIO driver – a reusable pin abstraction over the generated register layer (Interfaces.ESP32S3.GPIO + Interfaces.ESP32S3.IO_MUX).

Any pad 0 .. 48: configure direction / pull / drive strength, and set / clear / toggle / write / read. (Pin interrupts are a follow-up.)

Task-safe: Set / Clear / Write use the hardware-atomic W1TS/W1TC banks and Read is a pure load, so those are safe to call concurrently as-is. The read-modify-write operations (Configure, Toggle) are serialised through a protected object. (Holding a protected object, this package requires a tasking runtime.) Driving the same pin from two tasks is still the app's call – the lock keeps the registers consistent, not your intent.

Pads that the silicon does not expose, or that are bonded to the in-package SPI flash / octal PSRAM, are

excluded by the Pin_Id subtype below: driving them would hang the chip, so naming one is a compile-time error for a static value and a predicate check at run time.

Public API

```
   --  GPIO pad numbers as the silicon numbers them (0 .. 48); -1 is the "no pin"
   --  sentinel that drivers use for an optional line left unrouted (Optional_Pin).
   type Pad_Number is range -1 .. 48;

   --  Sentinel for an optional pin argument (e.g. an unused SPI chip-select).
   No_Pin : constant Pad_Number := -1;

   --  A GPIO pin an application may legitimately drive -- the type EVERY driver
   --  uses to name a pin.  The static predicate excludes the pads that don't
   --  exist on the ESP32-S3 (22 .. 25) and those wired to the in-package SPI
   --  flash and octal PSRAM (26 .. 37); driving any of them hangs the chip.
   --  (Boards without octal PSRAM could also use 33 .. 37, but the bare runtime
   --  here maps octal PSRAM, so they are reserved.)
   --
   --  Enforcement: because the predicate is STATIC, a reserved/absent pad used
   --  where a Pin_Id is expected as a static value is flagged at COMPILE time
   --  ("static expression fails static predicate check") -- so declare pin
   --  constants of type Pin_Id to get that check.  A dynamic value is verified
   --  by the subtype's predicate at RUN time wherever assertion/predicate checks
   --  are enabled (-gnata).  (No Predicate_Failure message: it would pull in
   --  Ada.Exceptions, which the light-tasking runtime does not provide.)
   subtype Pin_Id is Pad_Number range 0 .. 48 with
     Static_Predicate => Pin_Id in 0 .. 21 | 38 .. 48;

   --  A pin argument that may be omitted: a real Pin_Id, or No_Pin (-1).
   subtype Optional_Pin is Pad_Number with
     Static_Predicate => Optional_Pin in -1 | 0 .. 21 | 38 .. 48;

   type Pin_Mode  is (Input, Output);
   type Pull_Mode is (Floating, Pull_Up, Pull_Down);

   --  IO_MUX FUN_DRV field (0 .. 3), roughly 5 / 10 / 20 / 40 mA.
   type Drive_Strength is (Drive_Weak, Drive_Medium, Drive_Strong, Drive_Strongest);

   --  Configure a pad as a plain GPIO: direction + pull + drive. The pad is
   --  always routed through the GPIO matrix as a software-controlled GPIO
   --  (IO_MUX MCU_SEL = 1, GPIO output index 256). Output pads get their driver
   --  enabled; input pads get the input buffer enabled. (Routing a pad to a
   --  peripheral signal is the job of that peripheral's own Configure_Pins,
   --  which programs the matrix directly -- not this driver.)
   procedure Configure
     (Pin   : Pin_Id;
      Mode  : Pin_Mode;
      Pull  : Pull_Mode      := Floating;
      Drive : Drive_Strength := Drive_Medium);

   procedure Set    (Pin : Pin_Id);              --  drive high  (atomic W1TS)
   procedure Clear  (Pin : Pin_Id);              --  drive low   (atomic W1TC)
```

```
   procedure Toggle (Pin : Pin_Id);              --  flip the current output level
   procedure Write  (Pin : Pin_Id; On : Boolean);
   function  Read   (Pin : Pin_Id) return Boolean;  --  sample the input level
```

ESP32S3.GPIO.Interrupts

Source: src/esp32s3-gpio-interrupts.ads

GPIO pin interrupts for the ESP32-S3 bare-metal (Jorvik) runtime.

The GPIO peripheral has one interrupt SOURCE (the OR of all pins' latched status); this module routes it through the interrupt matrix to the runtime's level-3 device slot (Ada.Interrupts.Names.Device_L3_0 = CPU_INT 23) and owns the ISR there. The ISR demuxes by GPIO status and runs your per-pin callback.

Ravenscar/Jorvik attaches handlers statically, so the application does not pass an ISR – it registers a per-pin Callback that this module's ISR calls. That Callback runs in INTERRUPT context (inside a protected action at the level-3 ceiling): keep it short, and don't call a lower-ceiling protected object or block. The usual idiom is to bump an Atomic flag or Set a Suspension_Object that a normal task is waiting on, and do the real work there.

Requires a tasking runtime.

Public API

```
   --  What raises the interrupt on the pin.
   type Trigger is
     (Rising_Edge, Falling_Edge, Any_Edge, Low_Level, High_Level);

   --  Per-pin action, run in interrupt context (see the note above).
   type Callback is access procedure;

   --  Enable Pin's interrupt with the given trigger and action.  On first use
   --  this also routes the GPIO source to the level-3 device slot.  The pin's
   --  input buffer must be on (ESP32S3.GPIO.Configure already enables it).
   procedure Enable (Pin : Pin_Id; On : Trigger; Action : Callback);

   --  Stop delivering Pin's interrupt.
   procedure Disable (Pin : Pin_Id);
```

ESP32S3.GPS

Source: src/esp32s3-gps.ads

Generic NMEA-0183 GPS receiver driver (UART), task-driven.

Unlike the passive I2C device drivers (ESP32S3.PCF85063A / .QMI8658C, which you poll through a Device handle), this is a SINGLETON background SERVICE: a library-level task owns one UART for its lifetime, continuously reads the receiver's NMEA stream, decodes it, and publishes the results into a PROTECTED store. The application just reads that store – there is no Device handle.

```
Wiring (all three pins optional; Rx is the only one needed to receive):
   Rx   <- the receiver's TXD  (data IN from the GPS)
   Tx   -> the receiver's RXD  (data OUT to the GPS; for config -- see below)
   Pps  <- the receiver's 1PPS (a GPIO interrupt, for time alignment)
```

Concurrency / tearing: every published value is written and read under the protected store's lock, so a reader never sees a half-updated value. In particular Latitude and Longitude are ONE record (Position), updated by a single protected action, so a fix is always a consistent pair.

Staleness: each value group carries the Ada.Real_Time.Time at which it was last refreshed. The driver only refreshes a group from a VALID sentence (a lost fix is not written), so a stale group keeps its old timestamp – compare Age (R.Updated_At) against your tolerance to decide whether to trust it.

Uses the controlled UART Session + a task + protected objects => embedded / full profiles only (excluded from light-tasking, like the other Session drivers). Call Setup once at startup.

Public API

```
   ------------------------------------------------------------------------
   --  Decoded quantities.
   ------------------------------------------------------------------------

   --  Latitude / longitude in 1e-7 degrees (the u-blox convention): exact,
   --  integer, and atomically paired.  +N / -S, +E / -W.
   type Position is record
      Latitude  : Interfaces.Integer_32 := 0;
      Longitude : Interfaces.Integer_32 := 0;
```

ESP32S3.GPS.L76K

Source: `src/esp32s3-gps-l76k.ads`

Quectel L76K proprietary PCAS commands (NMEA-framed vendor messages).

These configure the receiver by SENDING sentences to it (via the parent's ESP32S3.GPS.Send, so a Tx pin must be routed in Setup). They are SPECIFIC TO THE L76K: only `with` and call this package when the module on the bus is an L76K (the package itself is the "L76K only" gate). Reference: Quectel L76K GNSS Protocol Specification V1.1, section 2.3.

Status: every PCAS command is implemented, but only Set_Constellation (PCAS04) has been exercised on hardware – the others are provided for completeness and are UNTESTED.

Public API

```
   ------------------------------------------------------------------------
   --  PCAS04 -- GNSS constellation selection (TESTED).  QZSS is always on.
   ------------------------------------------------------------------------

   type Constellation is
     (GPS_Only,         --  mode 1
      BeiDou_Only,      --  mode 2
      GPS_BeiDou,       --  mode 3 (the L76K default)
      GLONASS_Only,     --  mode 4
      GPS_GLONASS,      --  mode 5
      BeiDou_GLONASS,   --  mode 6
      GPS_BeiDou_GLONASS);  --  mode 7

   --  $PCAS04,<mode> -- choose which constellations the receiver searches.
   procedure Set_Constellation (Config : Constellation);
```

```
   ------------------------------------------------------------------------
   --  The remaining PCAS commands are implemented but UNTESTED.
   ------------------------------------------------------------------------

   --  PCAS01 -- NMEA port baud rate.  NOTE: after this takes effect the receiver
   --  talks at the new rate; reconfigure the host UART (ESP32S3.GPS only sets
   --  the rate at Setup) to keep receiving.
   type Baud_Setting is
     (B_4800, B_9600, B_19200, B_38400, B_57600, B_115200);  --  modes 0..5
   procedure Set_Baud_Rate (Rate : Baud_Setting);   --  $PCAS01,<n>  UNTESTED

   --  PCAS02 -- positioning (fix) rate.  Rates above 1 Hz require single NMEA
   --  output and 115200 baud (datasheet note).
   type Update_Rate is (Rate_1Hz, Rate_2Hz, Rate_5Hz);
   procedure Set_Update_Rate (Rate : Update_Rate); --  $PCAS02,<ms>  UNTESTED

   --  PCAS03 -- per-sentence output rate: 0 = off, 1 .. 9 = once every N fixes.
   subtype Output_Rate is Natural range 0 .. 9;
   procedure Set_NMEA_Output                        --  $PCAS03,...   UNTESTED
     (GGA, GLL, GSA, GSV, RMC, VTG, ZDA, ANT : Output_Rate := 1);

   --  PCAS10 -- restart the receiver.
   type Restart_Mode is (Hot, Warm, Cold, Cold_Factory);  --  flags 0..3
   procedure Restart (Mode : Restart_Mode);         --  $PCAS10,<n>  UNTESTED
```

ESP32S3.GPS.NMEA *(internal)*

Source: `src/esp32s3-gps-nmea.ads`

NMEA-0183 sentence decoding for ESP32S3.GPS – pure logic, no UART, no tasking, no storage, so it is directly testable (mirrors the ESP32S3. / .Engine split). Recognises GGA (fix + position + altitude + satellites), RMC (position + validity + velocity + date/time), ZDA (UTC time + date, available even before a position fix), GLL (position + time), VTG (velocity), GSV (satellites in view + C/N0), and GSA (2D/3D mode + DOP).

Parse validates the XOR checksum first; on a bad checksum or an unrecognised talker it reports Recognised = False and the Has_* flags stay clear. Each Has_* flag says whether that field was present and decoded, so the caller publishes only what actually arrived.

Public API

```
   --  Everything one GGA or RMC sentence can yield.  Only the fields whose Has_*
   --  flag is set are meaningful.
   type Parsed is record
      Recognised   : Boolean := False;   --  checksum OK and a GGA/RMC sentence

      Has_Position : Boolean := False;
      Pos          : Position;

      Fix_Valid    : Boolean := False;   --  RMC status 'A' / GGA quality > 0
      Has_Quality  : Boolean := False;
      Quality      : Fix_Quality := No_Fix;
      Has_Sats     : Boolean := False;
      Satellites   : Natural := 0;
```

```
      Has_Altitude : Boolean := False;
      Altitude_MM  : Integer := 0;

      Has_Time     : Boolean := False;
      Time         : UTC_Time;
      Has_Date     : Boolean := False;
      Day          : Date;

      Has_Velocity : Boolean := False;
      Speed_MMS    : Natural := 0;
      Course_CDeg  : Natural := 0;

      Has_Sky      : Boolean := False;   --  GSV
      In_View      : Natural := 0;
      Max_SNR      : Natural := 0;
      Sat_Count    : Natural := 0;       --  satellites in THIS GSV message (0..4)
      Sats         : Satellite_List (1 .. 4) := (others => <>);

      Has_DOP      : Boolean := False;   --  GSA
      Used         : Natural := 0;
      Mode         : Fix_Type := Fix_None;
      PDOP_C       : Natural := 0;
      HDOP_C       : Natural := 0;
      VDOP_C       : Natural := 0;
```

ESP32S3.I2C

Source: *src/esp32s3-i2c.ads*

ESP32-S3 I2C master (I2C0 / I2C1), task-safe.

This is the ONLY I2C interface the application sees. The raw register driver lives in the private child ESP32S3.I2C.Engine (un-`with`-able from outside this subtree), so the unsynchronised "ZFP" primitives can't be called by accident – access is always mediated here.

Each host is guarded by a protected object; Acquire hands out a limited, non-copyable Session that owns the host exclusively (other tasks suspend on Acquire until it is released). The blocking transaction (Write / Read) runs OUTSIDE the protected lock – the lock only arbitrates ownership, never held across the bus busy-wait.

This first cut is a polled, single-segment master: each Write / Read is one complete START..STOP transaction, up to 32 data bytes (the hardware FIFO depth). Requires a tasking runtime (Jorvik light-tasking or richer).

Public API

```
   --  The two general-purpose I2C controllers.
   type I2C_Host is (I2C0, I2C1);

   --  7-bit slave address (the R/W bit is added by the driver).
   subtype Slave_Address is Natural range 0 .. 16#7F#;

   type Byte is new Interfaces.Unsigned_8;
   type Byte_Array is array (Natural range <>) of Byte;

   --  Largest single-transaction payload (the controller's TX/RX FIFO depth).
```

```
   Max_Transfer : constant := 32;

   --  An exclusive hold on a host.  Limited (cannot be copied) and CONTROLLED:
   --  releases the host automatically on scope exit, including during exception
   --  unwinding, so a fault between Acquire and Release can't leak the lock.
   --  Release stays available to hand the host back early (idempotent).  This
   --  relies on finalization -> these task-safe drivers target embedded/full.
   type Session is limited private;

   ------------------------------------------------------------------------------
   --  One-time host configuration -- call once per host at startup, before any
   --  task contends for it (single-threaded).
   ------------------------------------------------------------------------------

   --  Bring Host up as a master at the given SCL bit clock (Hz, standard 100
   --  kHz / fast 400 kHz are typical; clamped to a sane range).
   procedure Setup (Host : I2C_Host; Clock_Hz : Positive := 100_000);

   --  Route the host's SCL/SDA to physical pads as open-drain lines with the
   --  internal pull-ups enabled (no external resistors needed for a quick
   --  bring-up; add real pull-ups for production buses).  Scl/Sda are validated
   --  GPIO pins -- a reserved/absent pad is rejected at compile or run time.
   procedure Configure_Pins
     (Host : I2C_Host; Scl : ESP32S3.GPIO.Pin_Id; Sda : ESP32S3.GPIO.Pin_Id);

   ------------------------------------------------------------------------------
   --  Concurrent, mutually-exclusive use.
   ------------------------------------------------------------------------------

   --  Raised by Acquire if Host was never Setup -- configuration must precede
   --  ownership (see the one-time configuration section above).
   Not_Initialized : exception;

   --  Raised by Write/Read if the Session does not currently hold a host.  Both
   --  reach the hardware only through one ownership-checked gateway in the body,
   --  so "transact without holding the host" fails loudly.
   Not_Owned : exception;

   --  Take exclusive ownership of a Setup host.  Suspends until no other task
   --  holds it.  Keep it across a whole transaction, then Release.  Raises
   --  Not_Initialized if Host was never Setup.
   procedure Acquire (S : in out Session; Host : I2C_Host);

   --  Master write: START, (Addr<<1 | W), Data bytes, STOP.  Success is True
   --  iff the slave ACKed the address and every byte.  Data length 0 sends an
   --  address-only probe (useful for bus scanning).  Blocking.  Raises
   --  Not_Owned unless S currently holds a host.
   procedure Write (S        : Session;
                    Addr     : Slave_Address;
                    Data     : Byte_Array;
                    Success  : out Boolean;
                    Check_Ack : Boolean := True);
```

```
   --  Master read: START, (Addr<<1 | R), read Data'Length bytes (ACK all but
   --  the last, NACK the last), STOP.  Success is True iff the slave ACKed the
   --  address.  Blocking.  Raises Not_Owned unless S currently holds a host.
   procedure Read (S       : Session;
                   Addr    : Slave_Address;
                   Data    : out Byte_Array;
                   Success : out Boolean);

   --  Relinquish ownership (lets a waiting task proceed).  Harmless if already
   --  released.  Always release a Session you Acquired.
   procedure Release (S : in out Session);
```

ESP32S3.I2C.Engine *(internal)*

Source: src/esp32s3-i2c-engine.ads

RAW I2C0/I2C1 master register driver – the ZFP-safe *mechanism* with NO mutual exclusion. PRIVATE child: only the ESP32S3.I2C subtree may use it; the application cannot `with` it, and reaches I2C only through the task-safe parent (ESP32S3.I2C). See the parent for the design rationale.

(I2C_Host / Slave_Address / Byte / Byte_Array are declared in the parent and used here by child visibility.)

Public API

```
   --  A configured master controller.
   type Bus is private;

   function Open (Host : I2C_Host; Clock_Hz : Positive) return Bus;

   function Is_Open (B : Bus) return Boolean;

   --  Route SCL/SDA to pads as open-drain with internal pull-ups.
   procedure Configure_Pins
     (B : Bus; Scl : ESP32S3.GPIO.Pin_Id; Sda : ESP32S3.GPIO.Pin_Id);

   --  One START..STOP master write transaction.  Success := slave ACKed addr +
   --  every data byte (when Check_Ack).  Data length 0 is an address-only probe.
   procedure Write (B       : Bus;
                    Addr    : Slave_Address;
                    Data    : Byte_Array;
                    Success : out Boolean;
                    Check_Ack : Boolean := True);

   --  One START..STOP master read transaction (ACK all but last byte).
   --  Success := slave ACKed the address.
   procedure Read (B       : Bus;
                   Addr    : Slave_Address;
                   Data    : out Byte_Array;
                   Success : out Boolean);

   procedure Close (B : in out Bus);
```

ESP32S3.I2S

Source: `src/esp32s3-i2s.ads`

ESP32-S3 I2S (digital audio serial bus), task-safe, DMA-driven.

Two instances (I2S0 / I2S1). Each moves PCM samples over the GDMA crossbar (the S3 I2S has no CPU FIFO – data flows only through DMA), in standard Philips/TDM format. This driver brings a port up as a master that generates BCLK + WS and shifts a sample buffer out on its data-out line (TX) and/or captures one on its data-in line (RX).

Like ESP32S3.SPI, the raw register driver lives in the private child ESP32S3.I2S.Engine; the application only ever sees this package. Each port is guarded by a protected object, and Acquire hands out a limited, non-copyable, CONTROLLED Session that owns the port exclusively and releases it automatically on scope exit (so a fault between Acquire and Release can't leak the lock). The blocking DMA transfer runs OUTSIDE the lock. Relies on finalization, so it targets the embedded/full profile.

Requires a tasking runtime (Jorvik light-tasking or richer).

Public API

```
   --  The two I2S controllers.
   type I2S_Port is (I2S0, I2S1);

   --  Sample width in bits (the buffer element size the DMA moves).
   type Sample_Bits is (Bits_8, Bits_16, Bits_24, Bits_32);

   --  Data-path mode.
   --
   --  Standard : ordinary I2S/TDM -- the PCM buffer appears verbatim on the
   --             data wire (BCLK + WS framed), for a codec/DAC/ADC.
   --
   --  PDM      : the hardware sigma-delta converters sit between the buffer and
   --             the wire.  On TX the PCM2PDM converter turns each PCM sample
   --             into a 1-bit pulse-density stream (feed it to a class-D amp or
   --             an RC low-pass for analog out); on RX the PDM2PCM converter
   --             decimates a 1-bit PDM input back to PCM (a PDM microphone, or a
   --             PDM-output ADC).  The DMA still moves ordinary PCM either way,
   --             so Write/Read/Transfer are unchanged -- only the on-wire format
   --             differs.  Note the converters high-pass filter (remove DC), so
   --             a constant level does not survive a PDM round trip.
   type I2S_Mode is (Standard, PDM);

   --  Sentinel for Configure_Pins: leave that line unrouted.
   No_Pin : constant ESP32S3.GPIO.Pad_Number := ESP32S3.GPIO.No_Pin;

   --  An exclusive hold on a port.  Limited (cannot be copied) and CONTROLLED
   --  (auto-releases on scope exit, including during exception unwinding).
   type Session is limited private;

   ---------------------------------------------------------------------------
   --  One-time port configuration -- call once per port at startup, before any
   --  task contends for it (single-threaded).
   ---------------------------------------------------------------------------

   --  Bring Port up as a stereo master at (about) Sample_Rate_Hz with the given
```

```
   --  sample width and data-path mode, and Claim its GDMA channel.  In Standard
   --  mode BCLK = Sample_Rate * Bits * 2; in PDM mode the serial clock runs at
   --  Sample_Rate * 128 (the sigma-delta oversample).
   procedure Setup (Port        : I2S_Port;
                    Sample_Rate : Positive    := 16_000;
                    Bits        : Sample_Bits := Bits_16;
                    Mode        : I2S_Mode    := Standard);

   --  Internal data-line loopback through one GPIO pad (self-test; no wiring):
   --  TX and RX share WS+BCK internally (the hardware SIG_LOOPBACK bit) and the
   --  data-out signal is fed back into data-in on Pad.
   procedure Enable_Loopback (Port : I2S_Port; Pad : ESP32S3.GPIO.Pin_Id);

   --  Route the port's signals to physical pads for an external codec.  Each
   --  line is a validated GPIO pin; pass No_Pin to leave a line unrouted (e.g.
   --  Din for a TX-only DAC, or Dout for an RX-only microphone).
   --  Mclk routes the master clock out (e.g. to a codec's MCLK input); only
   --  supported on I2S0.  Leave No_Pin for codecs that clock from BCLK.
   procedure Configure_Pins (Port : I2S_Port;
                             Bclk : ESP32S3.GPIO.Optional_Pin := No_Pin;
                             Ws   : ESP32S3.GPIO.Optional_Pin := No_Pin;
                             Dout : ESP32S3.GPIO.Optional_Pin := No_Pin;
                             Din  : ESP32S3.GPIO.Optional_Pin := No_Pin;
                             Mclk : ESP32S3.GPIO.Optional_Pin := No_Pin);

   ------------------------------------------------------------------------------
   --  Concurrent, mutually-exclusive use.
   ------------------------------------------------------------------------------

   --  Raised by Acquire if Port was never Setup -- configuration must precede
   --  ownership (see the one-time configuration section above).
   Not_Initialized : exception;

   --  Raised by Write/Read/Transfer if the Session does not currently hold a
   --  port.  All reach the hardware only through one ownership-checked gateway
   --  in the body, so "transfer without holding the port" fails loudly.
   Not_Owned : exception;

   --  Take exclusive ownership of a Setup port (suspends until it is free).
   --  Raises Not_Initialized if Port was never Setup.
   procedure Acquire (S : in out Session; Port : I2S_Port);

   --  Shift Length bytes (1 .. 4095) from Tx out on the data-out line.  Blocking.
   --  Buffer in internal SRAM.  Raises Not_Owned unless S holds a port.
   procedure Write (S : Session; Tx : System.Address; Length : Natural);

   --  Capture Length bytes (1 .. 4095) from the data-in line into Rx.  Blocking.
   --  Raises Not_Owned unless S holds a port.
   procedure Read (S : Session; Rx : System.Address; Length : Natural);

   --  Full-duplex: shift Tx out and capture Rx in simultaneously (same Length).
   --  Raises Not_Owned unless S holds a port.
   procedure Transfer (S : Session; Tx, Rx : System.Address; Length : Natural);
```

```
   --  Start streaming Tx (Length bytes, 1 .. 4095) on a self-looping DMA and
   --  return immediately, leaving the TX clock running: the buffer is replayed
   --  forever with NO inter-buffer gap -- gapless playback of a periodic
   --  waveform with zero CPU cost after the call.  Tx must stay valid (in
   --  internal SRAM) and should hold a whole number of wave periods so the
   --  wrap is seamless.  Stop halts it.  Raises Not_Owned unless S holds a port.
   procedure Start_Continuous (S : Session; Tx : System.Address; Length : Natural);

   --  Stop a continuous transmit started by Start_Continuous (TX clock off).
   --  Raises Not_Owned unless S holds a port.
   procedure Stop (S : Session);

   --  Relinquish ownership (lets a waiting task proceed).  Idempotent.
   procedure Release (S : in out Session);
```

ESP32S3.I2S.Engine *(internal)*

Source: *src/esp32s3-i2s-engine.ads*

RAW I2S0/I2S1 register driver – the ZFP-safe *mechanism* with NO mutual exclusion. PRIVATE child: only the ESP32S3.I2S subtree may use it. See the parent (ESP32S3.I2S) for the design rationale.

Public API

```
   --  A configured port plus its Claimed GDMA channel.  Limited because it holds
   --  a (limited, controlled) GDMA Channel; built in place by Open.
   type Bus is limited private;

   procedure Open (B           : in out Bus;
                   Port        : I2S_Port;
                   Sample_Rate : Positive;
                   Bits        : Sample_Bits;
                   Mode        : I2S_Mode);

   function Is_Open (B : Bus) return Boolean;

   procedure Configure_Pins (B : Bus;
                             Bclk : ESP32S3.GPIO.Optional_Pin;
                             Ws   : ESP32S3.GPIO.Optional_Pin;
                             Dout : ESP32S3.GPIO.Optional_Pin := No_Pin;
                             Din  : ESP32S3.GPIO.Optional_Pin := No_Pin;
                             Mclk : ESP32S3.GPIO.Optional_Pin := No_Pin);

   procedure Enable_Loopback (B : Bus; Pad : ESP32S3.GPIO.Pin_Id);

   procedure Write    (B : Bus; Tx : System.Address; Length : Natural);
   procedure Read     (B : Bus; Rx : System.Address; Length : Natural);
   procedure Transfer (B : Bus; Tx, Rx : System.Address; Length : Natural);

   --  Start the TX path streaming Tx (Length bytes) on a SELF-LOOPING DMA and
   --  leave it running: the buffer is replayed forever with no inter-buffer
   --  gap (gapless).  Returns immediately; Stop halts it.  Tx in internal SRAM,
```

```
   --  Length 1 .. 4095, and Tx should hold a whole number of wave periods.
   procedure Start_Continuous (B : Bus; Tx : System.Address; Length : Natural);

   --  Stop a continuous transmit (TX clock off).
   procedure Stop (B : Bus);

   procedure Close (B : in out Bus);
```

ESP32S3.LCD

Source: `src/esp32s3-lcd.ads`

ESP32-S3 LCD (the LCD half of the LCD_CAM controller) – 8-bit Intel-8080 parallel master, DMA-driven.

Drives an 8-bit "i80"/MCU parallel display (or any 8-bit parallel sink): a byte buffer is streamed out the data bus, one byte per pixel clock (PCLK), over the GDMA crossbar. (The camera-receive half and the RGB/16-bit modes are not covered here.)

The single controller is guarded by a protected object; Acquire hands out a limited, controlled Session that owns it exclusively and releases on scope exit. Uses finalization, so it targets the embedded/full profile.

Public API

```
   No_Pin : constant ESP32S3.GPIO.Pad_Number := ESP32S3.GPIO.No_Pin;

   --  The eight parallel data lines (D0 .. D7); any may be left unrouted.
   type Data_Pins is array (0 .. 7) of ESP32S3.GPIO.Optional_Pin;

   type Session is limited private;

   ------------------------------------------------------------------------
   --  One-time configuration (single-threaded at startup).  This is the only
   --  port-based call; pin routing and clock-out are post-Setup and require the
   --  held controller (see below).
   ------------------------------------------------------------------------

   --  Bring the LCD up in 8-bit mode at (about) Pclk_Hz pixel clock and Claim its
   --  GDMA channel.  Pixel clock = 20 MHz / round(20 MHz / Pclk_Hz).
   procedure Setup (Pclk_Hz : Positive := 1_000_000);

   ------------------------------------------------------------------------
   --  Concurrent, mutually-exclusive use.  Acquire the controller, then run
   --  every transfer AND every post-Setup reconfiguration through it -- so
   --  changing a setting requires ownership and can never race another task.
   --  All register access lives in the private Engine child; the handle is
   --  hidden in the body and reached only through one ownership-checked gateway.
   ------------------------------------------------------------------------

   --  Raised by Acquire if the controller was never Setup (or its GDMA channel
   --  could not be claimed) -- configuration must precede ownership.
   Not_Initialized : exception;

   --  Raised by any operation below if S does not hold the controller.  Each
   --  reaches the hardware only through the gateway, so "use it without holding
```

```
   --  it" fails loudly.
   Not_Owned : exception;

   --  Take exclusive ownership of the Setup controller (suspends until free).
   --  Raises Not_Initialized if Setup did not succeed.
   procedure Acquire (S : in out Session);

   --  Route the data bus and pixel clock to physical pads (for a real display),
   --  on the held controller.  Raises Not_Owned unless S holds it.
   procedure Configure_Pins (S : Session; Data : Data_Pins;
                             Pclk : ESP32S3.GPIO.Optional_Pin);

   --  Free-run the pixel clock continuously on Pclk_Pad (no data transaction) --
   --  useful as a bus clock and for verifying the clock on its own.  On the held
   --  controller; raises Not_Owned unless S holds it.
   procedure Enable_Clock_Out (S : Session; Pclk_Pad : ESP32S3.GPIO.Pin_Id);

   --  Stream Length bytes (1 .. 4095) from Tx out the data bus, one per PCLK.
   --  Blocking; Ok is True once the transfer completes.  Buffer in internal SRAM.
   --  Raises Not_Owned unless S holds the controller.
   procedure Transmit (S : Session; Tx : System.Address; Length : Natural;
                       Ok : out Boolean);

   procedure Release (S : in out Session);
```

ESP32S3.LCD.Engine *(internal)*

Source: src/esp32s3-lcd-engine.ads

RAW LCD_CAM (i80 parallel) register driver – the ZFP-safe *mechanism* with NO mutual exclusion. PRIVATE child: only the ESP32S3.LCD subtree may use it; the application reaches LCD only through the task-safe parent, which hides the Bus handle and hands it out solely through its ownership-checked gateway. This is the only unit that names the LCD_CAM registers and owns the GDMA channel.

(Data_Pins is declared in the parent and used here by child visibility.)

Public API

```
   --  A configured controller: its claimed GDMA channel + a validity bit.
   --  Limited because it holds a limited-controlled GDMA Channel.
   type Bus is limited private;

   --  Bring the LCD up in 8-bit mode at (about) Pclk_Hz and Claim its GDMA
   --  channel.  Is_Valid is False if the channel could not be claimed.
   procedure Open (B : in out Bus; Pclk_Hz : Positive);
   function  Is_Valid (B : Bus) return Boolean;

   --  Route the data bus and pixel clock to physical pads.
   procedure Configure_Pins (B    : Bus;
                             Data : Data_Pins;
                             Pclk : ESP32S3.GPIO.Optional_Pin);

   --  Free-run the pixel clock continuously on Pclk_Pad (no data transaction).
```

```
procedure Enable_Clock_Out (B : Bus; Pclk_Pad : ESP32S3.GPIO.Pin_Id);

--  Stream Length bytes (1 .. 4095) from Tx out the data bus, one per PCLK.
--  Ok is True once the transfer completes.
procedure Transmit (B : Bus; Tx : System.Address; Length : Natural;
                    Ok : out Boolean);
```

ESP32S3.LEDC

Source: *src/esp32s3-ledc.ads*

ESP32-S3 LEDC (LED PWM controller) – up to eight independent PWM outputs.

The S3 LEDC has eight low-speed channels fed by four timers; a channel picks a timer (which sets its frequency + duty resolution) and drives a GPIO with a duty cycle you can change at run time. Unlike MCPWM (motor control: dead-time, fault, capture), LEDC is the simple "dim an LED / generate a clean PWM" block.

Ownership: a channel is a shared resource, so it is handed out as a CLAIMED handle. A Channel handle is LIMITED (non-copyable – two tasks can't drive the same channel, nor reuse one through a stale copy) and CONTROLLED (it stops the output and releases the channel automatically on scope exit, including on an exception). Because you exclusively own a claimed channel, Set_Duty needs no lock. Uses finalization, so it targets the embedded/full profile.

Public API

```
type Channel_Index is range 0 .. 7;        --  the eight LEDC channels

subtype Resolution is Positive range 1 .. 14;   --  duty-cycle bits
subtype Duty_Percent is Float range 0.0 .. 100.0;

--  Opaque, non-copyable handle to a claimed channel.  Default-initialised
--  invalid (check Is_Valid); auto-releases on scope exit.
type Channel is limited private;

--  Claim channel Index into C.  If it is already claimed, C is left invalid
--  (Is_Valid False).  (If C already holds a channel it is released first.)
procedure Claim (C : in out Channel; Index : Channel_Index);

--  True when Claim succeeded (a real channel is held).
function Is_Valid (C : Channel) return Boolean;

--  Stop the output and return the channel to the free pool.  Harmless on an
--  invalid handle.
procedure Release (C : in out Channel);

--  Configure C for PWM at Freq Hz with Bits of duty resolution, routed to Pin.
--  Duty starts at 0 %.  NOTE: a channel uses timer (Index mod 4), so channels
--  whose indices differ by 4 (e.g. 0 and 4) share a timer and therefore one
--  frequency/resolution -- use channels 0 .. 3 for four independent frequencies.
--  The achievable frequency depends on Bits: freq_max = 80 MHz / 2**Bits.
procedure Configure (C    : in out Channel;
                     Freq : Positive;
                     Pin  : ESP32S3.GPIO.Pin_Id;
```

```
                      Bits : Resolution := 10);

   --  Set C's duty cycle (0 .. 100 %).  Takes effect at the next period; safe
   --  without a lock because you exclusively own C.
   procedure Set_Duty (C : Channel; Percent : Duty_Percent);

   --  Force the output inactive (low) without releasing the channel.
   procedure Stop (C : Channel);
```

ESP32S3.MCPWM

Source: *src/esp32s3-mcpwm.ads*

ESP32-S3 MCPWM (Motor-Control PWM) – edge-aligned PWM output.

Two units (MCPWM0 / MCPWM1), each with three independent generator channels and three capture channels. A generator channel is one timer + one operator producing a single edge-aligned PWM on output A, routed to a GPIO: high at the start of each period, low when the up-counting timer reaches the duty comparator (duty = compare / period).

A channel can also drive a COMPLEMENTARY pair (the half-bridge / H-bridge motor-drive mode): the A output and an inverted B output, both from the same PWM, with programmable dead-time inserted between their edges so the two are never high together. Carrier (chopper) modulation, fault/trip-zone shutdown and edge capture are also provided.

Ownership / task-safety: a generator channel and a capture channel are shared hardware resources, so each is handed out as a CLAIMED handle (Channel / Capture). A handle is LIMITED (non-copyable – two tasks can't alias one channel, nor reuse one through a stale copy) and CONTROLLED (it releases its channel automatically on scope exit, including on an exception, stopping the generator's timer so a leaked handle can't keep driving a pad). Because you exclusively own a claimed channel, its operations (including Set_Duty) need no further locking. Using finalization, this driver targets the embedded/full profile (excluded from the light-tasking build).

Public API

```
   type MCPWM_Unit    is (MCPWM0, MCPWM1);
   type Channel_Index is (Ch0, Ch1, Ch2);    --  which generator channel of a unit

   subtype Duty_Percent is Float range 0.0 .. 100.0;

   --  Opaque, non-copyable handle to a claimed generator channel.  Default-
   --  initialised invalid (check Is_Valid); auto-releases on scope exit.
   type Channel is limited private;

   ------------------------------------------------------------------------
   --  Unit bring-up + channel ownership (single-threaded at startup for Setup;
   --  Claim/Release may run from any task -- a protected pool serialises them).
   ------------------------------------------------------------------------

   --  Raised by either Claim if Unit was never Setup -- the unit's clock must
   --  be brought up before any of its channels can be owned.
   Not_Initialized : exception;

   --  Bring the unit's clock up (PWM clock = 160 MHz).  Call once per unit,
```

```
   --  before claiming any of its channels or configuring its faults.
   procedure Setup (Unit : MCPWM_Unit);

   --  Claim generator channel Index of Unit into C.  If it is already claimed,
   --  C is left invalid (Is_Valid False).  (If C already holds a channel it is
   --  released first.)  C releases its channel automatically on scope exit --
   --  call Release only to hand it back early.  Raises Not_Initialized if Unit
   --  was never Setup.
   procedure Claim (C : in out Channel; Unit : MCPWM_Unit; Index : Channel_Index);

   --  True when Claim succeeded (a real channel is held).
   function Is_Valid (C : Channel) return Boolean;

   --  Return the channel to the free pool (stopping its timer first).  Harmless
   --  on an invalid handle.
   procedure Release (C : in out Channel);

   ------------------------------------------------------------------------------
   --  Per-channel configuration + run-time control (you must hold C).
   ------------------------------------------------------------------------------

   --  Configure C for edge-aligned PWM at Freq Hz (roughly 10 Hz .. 10 MHz; the
   --  divider is chosen automatically) and route its A output to Pin.  Duty
   --  starts at 0 %; the timer is left stopped -- call Start.
   --
   --  Complement_Pin (optional): also drive a complementary B output on that pad
   --  -- the inverse of A, with Dead_Time_Ns of dead-time inserted on each edge
   --  so A and B are never high simultaneously (half-bridge motor drive).
   procedure Configure_Channel
     (C    : Channel;
      Freq : Positive;
      Pin  : ESP32S3.GPIO.Pin_Id;
      Complement_Pin : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
      Dead_Time_Ns   : Natural := 0);

   --  Start / stop the channel's timer (the output halts in its current state).
   procedure Start (C : Channel);
   procedure Stop  (C : Channel);

   --  Set the channel's duty cycle (0 .. 100 %).  Single atomic register write;
   --  the new value is loaded glitch-free at the next period boundary.  Safe
   --  without a lock because you exclusively own C.
   procedure Set_Duty (C : Channel; Percent : Duty_Percent);

   ------------------------------------------------------------------------------
   --  Carrier (chopper) modulation -- chops the output with a high-frequency
   --  carrier (for transformer-coupled / isolated gate drives).  Carrier
   --  frequency = 160 MHz / (8 * (Prescale + 1)); Duty_Eighths is the carrier's
   --  own duty in 1/8ths; First_Pulse widens the first chop pulse (in carrier
   --  periods, 0 = same as the rest).
   ------------------------------------------------------------------------------
   subtype Carrier_Prescale is Natural range 0 .. 15;
   subtype Carrier_Duty     is Natural range 1 .. 7;
```

```
   subtype Carrier_Pulse    is Natural range 0 .. 15;

   procedure Set_Carrier (C : Channel;
                          Enable       : Boolean := True;
                          Prescale     : Carrier_Prescale := 0;
                          Duty_Eighths : Carrier_Duty := 4;
                          First_Pulse  : Carrier_Pulse := 1);

   ----------------------------------------------------------------------------
   --  Fault / trip-zone -- a fault input pin forces the outputs to a safe state
   --  (e.g. over-current shutdown).  Configure_Fault enables an input unit-wide
   --  (call after Setup); Protect_Channel makes a claimed channel react to it.
   ----------------------------------------------------------------------------
   type Fault_Input is (Fault0, Fault1, Fault2);
   type Fault_Mode  is (One_Shot, Cycle_By_Cycle);
   type Trip_Action is (No_Change, Force_Low, Force_High);

   --  Route Pin to a fault input and enable it (Active_High: a high level is the
   --  fault; else low).
   procedure Configure_Fault (Unit  : MCPWM_Unit;
                              Input  : Fault_Input;
                              Pin    : ESP32S3.GPIO.Pin_Id;
                              Active_High : Boolean := True);

   --  When Input faults, force channel C's A and B outputs to Action.  One_Shot
   --  latches until Clear_Fault; Cycle_By_Cycle holds only while the fault is
   --  asserted (re-evaluated each period).
   procedure Protect_Channel (C : Channel;
                              Input   : Fault_Input;
                              Mode    : Fault_Mode  := One_Shot;
                              Action  : Trip_Action := Force_Low);

   --  Clear a latched one-shot trip (re-enables the outputs if the fault is gone).
   procedure Clear_Fault (C : Channel);

   --  True while the channel is tripped (one-shot latched or cycle-by-cycle on).
   function Faulted (C : Channel) return Boolean;

   ----------------------------------------------------------------------------
   --  Capture -- timestamp edges on an input pin (measure an external signal's
   --  period / duty).  The capture timer runs on the APB clock.  A capture
   --  channel is claimed just like a generator channel.
   ----------------------------------------------------------------------------
   type Cap_Index is (Cap0, Cap1, Cap2);
   type Cap_Edge  is (Rising, Falling, Both_Edges);

   Capture_Clock_Hz : constant := 80_000_000;   --  APB clock the cap timer counts

   --  Opaque, non-copyable handle to a claimed capture channel.
   type Capture is limited private;

   --  Claim capture channel Index of Unit into Cap (see Claim for a Channel).
   --  Raises Not_Initialized if Unit was never Setup.
```

```
   procedure Claim (Cap : in out Capture; Unit : MCPWM_Unit; Index : Cap_Index);
   function Is_Valid (Cap : Capture) return Boolean;
   procedure Release (Cap : in out Capture);

   --  Route Pin to the claimed capture channel and start timestamping the given
   --  edges.
   procedure Configure_Capture (Cap : Capture;
                                Pin   : ESP32S3.GPIO.Pin_Id;
                                Edge  : Cap_Edge := Both_Edges);

   --  True once a new edge has been captured (and not yet read).
   function Capture_Pending (Cap : Capture) return Boolean;

   --  Read the latest capture timestamp (in Capture_Clock_Hz ticks) and which
   --  edge it was, and clear the pending flag.
   procedure Read_Capture (Cap : Capture;
                           Value   : out Interfaces.Unsigned_32;
                           Falling : out Boolean);
```

ESP32S3.PCF85063A

Source: *src/esp32s3-pcf85063a.ads*

NXP PCF85063A / PCF85063TP I2C real-time clock + calendar driver.

A small, fixed-address (0x51) RTC: seconds .. years in BCD, a programmable alarm (second / minute / hour / day / weekday, any subset), an oscillator- stop flag that reports whether time was kept across a power loss, and an active-low open-drain INT line (the alarm asserts it). Up to 400 kHz I2C.

This layers the chip's register protocol over the task-safe ESP32S3.I2C master. The driver hard-codes NO board wiring: you tell Setup which host and which SDA / SCL (and, optionally, which INT) pins the part is wired to, and the Device remembers them. Each operation then opens a short-lived I2C Session (a controlled type) for one complete transaction and lets it release the host automatically on scope exit – so concurrent callers serialise and a fault between acquire and release can't leak the bus. Needs the controlled Session => embedded / full profiles only (excluded from light-tasking, like the other Session drivers).

Register reads use the chip's auto-incrementing address pointer: a 1-byte write sets the pointer, a following read streams from it (the pointer survives the STOP between the two transactions).

```
Typical use:
   declare
      RTC : ESP32S3.PCF85063A.Device;
      T   : ESP32S3.PCF85063A.Time;
      Ok  : Boolean;
      St  : ESP32S3.PCF85063A.Status;
   begin
      --  state the wiring once -- here SDA = IO8, SCL = IO7, no INT line.
      ESP32S3.PCF85063A.Setup (RTC, Sda => 8, Scl => 7);
      ESP32S3.PCF85063A.Set_Time (RTC, (Year => 2026, Month => 6, Day => 22,
         Day_Of_Week => ESP32S3.PCF85063A.Monday,
         Hour => 14, Minute => 30, Second => 0), St);
      ESP32S3.PCF85063A.Get_Time (RTC, T, Ok, St);   --  Ok = clock integrity
   end;
```

Public API

```
   --  The PCF85063A has ONE fixed 7-bit bus address (no address pins).
   Bus_Address : constant ESP32S3.I2C.Slave_Address := 16#51#;

   ------------------------------------------------------------------------
   --  The calendar value the chip stores (BCD on the wire; binary here).
   ------------------------------------------------------------------------

   --  The chip keeps a 2-digit year (00..99); this driver anchors it to 2000.
   subtype Year_Number   is Natural range 2000 .. 2099;
   subtype Month_Number  is Natural range 1 .. 12;
   subtype Day_Number    is Natural range 1 .. 31;
   subtype Hour_Number   is Natural range 0 .. 23;   --  24-hour mode only
   subtype Minute_Number is Natural range 0 .. 59;
   subtype Second_Number is Natural range 0 .. 59;

   --  Day-of-week is just a free 0..6 counter in the chip; this naming is the
   --  driver's own convention (0 = Sunday) -- the hardware does not interpret it.
   type Weekday is
     (Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday);

   type Time is record
      Year        : Year_Number   := 2000;
      Month       : Month_Number  := 1;
      Day         : Day_Number    := 1;
      Day_Of_Week : Weekday       := Sunday;
      Hour        : Hour_Number   := 0;
      Minute      : Minute_Number := 0;
      Second      : Second_Number := 0;
```

ESP32S3.PCF85063A.Interrupts

Source: *src/esp32s3-pcf85063a-interrupts.ads*

The PCF85063A INT line, on the GPIO the device was Setup with.

INT is active-low and open-drain: it idles high (via a pull-up) and the chip pulls it low when the alarm fires (with the alarm interrupt enabled), holding it low until the alarm flag is cleared (PCF85063A.Acknowledge_Alarm). So the pin is configured as an input with the internal pull-up on, triggering on the FALLING edge.

These act on the INT pin stored in the Device by Setup – pass No_Pin there (a part with no INT connection) and Attach / Detach are no-ops.

The Action runs in interrupt context (see ESP32S3.GPIO.Interrupts): keep it short – set a Suspension_Object or bump an Atomic flag, then do the I2C work (reading status, acknowledging the alarm) in a normal task.

Public API

```
   --  The per-pin handler type (see ESP32S3.GPIO.Interrupts).
   subtype Callback is ESP32S3.GPIO.Interrupts.Callback;

   --  Configure Dev's INT pin as a pulled-up input and deliver a falling-edge
   --  interrupt to Action whenever INT asserts.  No-op if Dev was set up with
   --  No_Pin.  Routes the GPIO source to the runtime's level-3 device slot on
```

```
   --  first use (done by ESP32S3.GPIO.Interrupts).
   procedure Attach (Dev : Device; Action : Callback);

   --  Stop delivering Dev's INT interrupt.  No-op if Dev has no INT pin.
   procedure Detach (Dev : Device);
```

ESP32S3.PCNT

Source: *src/esp32s3-pcnt.ads*

ESP32-S3 PCNT (pulse counter) – count edges on an input signal.

The S3 has four counter units; each counts up (or up/down) as edges arrive on its input pin, into a signed 16-bit counter. The classic use is a quadrature / tachometer / flow-meter input. This driver exposes the common "count edges on a pin" case (the per-unit direction-control input and the threshold-event comparators are left at their pass-through defaults).

Ownership: a unit is a shared resource handed out as a CLAIMED handle, LIMITED (non-copyable) and CONTROLLED (released automatically on scope exit). Uses finalization, so it targets the embedded/full profile.

Public API

```
   type Unit_Index is range 0 .. 3;           --  the four counter units

   --  Non-copyable handle to a claimed unit (check Is_Valid after Claim).
   type Unit is limited private;

   procedure Claim (U : in out Unit; Index : Unit_Index);
   function Is_Valid (U : Unit) return Boolean;
   procedure Release (U : in out Unit);

   --  Route Pin to U's input and start counting.  By default each rising edge
   --  increments; set Both_Edges to count rising and falling edges.  The counter
   --  is cleared to 0.
   procedure Configure (U : in out Unit;
                        Pin : ESP32S3.GPIO.Pin_Id;
                        Both_Edges : Boolean := False);

   --  Current counter value (signed; wraps at +/- 32768).
   function Count (U : Unit) return Integer;

   --  Reset the counter to 0.
   procedure Clear (U : Unit);

   --  Pause / resume counting (the count is retained while paused).
   procedure Pause  (U : Unit);
   procedure Resume (U : Unit);
```

ESP32S3.QMI8658C

Source: *src/esp32s3-qmi8658c.ads*

QST QMI8658C 6-axis IMU (3-axis accelerometer + 3-axis gyroscope) I2C driver.

Same shape as ESP32S3.PCF85063A: the driver hard-codes no board wiring – you tell Setup which host and which SDA / SCL (and, optionally, which INT) pins the part is wired to, plus the I2C address (SA0 selects 0x6A / 0x6B), and the Device remembers them. Each operation then opens a short-lived I2C Session (a controlled type) for one complete transaction and lets it release the host automatically on scope exit – so concurrent callers serialise and a fault between acquire and release can't leak the bus. Needs the controlled Session => embedded / full profiles only (excluded from light-tasking).

Register reads use the chip's auto-incrementing address pointer (CTRL1.ADDR_AI, set by Configure): a 1-byte write sets the pointer, a following read streams from it. The driver runs the device little-endian (CTRL1.BE = 0).

```
Typical use:
   declare
      IMU : ESP32S3.QMI8658C.Device;
      A   : ESP32S3.QMI8658C.Axes;
      St  : ESP32S3.QMI8658C.Status;
   begin
      ESP32S3.QMI8658C.Setup (IMU, Sda => 8, Scl => 7);   --  state the wiring
      ESP32S3.QMI8658C.Reset (IMU, St);                   --  (then wait ~15 ms)
      ESP32S3.QMI8658C.Configure (IMU, St);               --  ranges + ODR + on
      ESP32S3.QMI8658C.Read_Accelerometer (IMU, A, St);   --  raw counts
      --  g = A.X / Float (ESP32S3.QMI8658C.Accel_LSB_Per_G (IMU))
   end;
```

Public API

```
   --  7-bit I2C address, selected by the SA0 / SDO pin.  SA0 has an internal
   --  200 k pull-up, so a floating pin reads high => 0x6A is the power-on default.
   Address_SA0_High : constant ESP32S3.I2C.Slave_Address := 16#6A#;    --  SA0 = VDDIO
   Address_SA0_Low  : constant ESP32S3.I2C.Slave_Address := 16#6B#;    --  SA0 = GND

   --  WHO_AM_I (register 0x00) returns this for every QST QMI8658.
   Who_Am_I_Value : constant Interfaces.Unsigned_8 := 16#05#;

   ------------------------------------------------------------------------
   --  Sensor configuration choices.
   ------------------------------------------------------------------------

   --  Accelerometer full scale (CTRL2.aFS).
   type Accel_Range is (Range_2G, Range_4G, Range_8G, Range_16G);

   --  Gyroscope full scale (CTRL3.gFS), in degrees per second.
   type Gyro_Range is
     (Range_16DPS,  Range_32DPS,   Range_64DPS,  Range_128DPS,
      Range_256DPS, Range_512DPS, Range_1024DPS, Range_2048DPS);

   --  Output data rate (CTRL2.aODR / CTRL3.gODR).  These are the 6DOF /
   --  gyroscope rates -- the rates in effect when both sensors are enabled, as
   --  Configure does.  (Accelerometer-only mode uses a different rate table; not
   --  exposed here.)
   type Output_Rate is
     (ODR_7520_Hz, ODR_3760_Hz, ODR_1880_Hz, ODR_940_Hz, ODR_470_Hz,
      ODR_235_Hz,  ODR_118_Hz,  ODR_59_Hz,   ODR_29_Hz);
```

```
   ------------------------------------------------------------------------
   --  Output data.
   ------------------------------------------------------------------------

   --  One 3-axis reading, raw 16-bit signed counts (sensor frame, right-handed).
   --  Convert to physical units with the *_LSB_* sensitivity below.
   type Axes is record
      X, Y, Z : Interfaces.Integer_16 := 0;
```

ESP32S3.QMI8658C.Interrupts

Source: *src/esp32s3-qmi8658c-interrupts.ads*

The QMI8658C INT line (INT1 / INT2), on the GPIO the device was Setup with.

The QMI8658C drives INT as a push-pull, active-high data-ready / event line by default, so the pin is configured as an input with a pull-down (idles low) and triggers on the RISING edge. (The on-chip polarity and which event drives the pin are programmable; adjust this child if you change them.)

These act on the INT pin stored in the Device by Setup – pass No_Pin there (no INT connection) and Attach / Detach are no-ops.

The Action runs in interrupt context (see ESP32S3.GPIO.Interrupts): keep it short – set a Suspension_Object or bump an Atomic flag, then do the I2C work (reading the samples / status) in a normal task.

Public API

```
   --  The per-pin handler type (see ESP32S3.GPIO.Interrupts).
   subtype Callback is ESP32S3.GPIO.Interrupts.Callback;

   --  Configure Dev's INT pin as a pulled-down input and deliver a rising-edge
   --  interrupt to Action whenever INT asserts.  No-op if Dev was set up with
   --  No_Pin.  Routes the GPIO source to the runtime's level-3 device slot on
   --  first use (done by ESP32S3.GPIO.Interrupts).
   procedure Attach (Dev : Device; Action : Callback);

   --  Stop delivering Dev's INT interrupt.  No-op if Dev has no INT pin.
   procedure Detach (Dev : Device);
```

ESP32S3.RMT

Source: *src/esp32s3-rmt.ads*

ESP32-S3 RMT (Remote Control / “infinitely-flexible pulse generator”).

RMT transmits and receives sequences of {level, duration} pulses – the workhorse for IR remotes, WS2812 (“NeoPixel”) LEDs, 1-Wire, and any custom bit-banged timing. The S3 has eight channels: 0 .. 3 transmit, 4 .. 7 receive, each with a 48-symbol RAM block. A pulse pair is one symbol (two {level, duration} fields); duration is in channel ticks, whose length you set via the channel’s resolution.

Ownership: a channel is a shared resource handed out as a CLAIMED handle. A handle is LIMITED (non-copyable) and CONTROLLED (releases its channel automatically on scope exit, including on an exception). TX and RX channels are distinct handle types so the two can’t be confused. Uses finalization, so it targets the embedded/full profile.

Public API

```
type TX_Index is range 0 .. 3;              --  the four transmit channels
type RX_Index is range 0 .. 3;              --  the four receive channels

--  A duration, in channel ticks (15-bit; tick length = 1 / Resolution_Hz).
type Tick_Count is range 0 .. 32_767;

--  One RMT symbol: two consecutive pulses (a level held for a duration).
type RMT_Symbol is record
   Level0    : Boolean    := False;
   Duration0 : Tick_Count := 0;
   Level1    : Boolean    := False;
   Duration1 : Tick_Count := 0;
```

ESP32S3.RTC

Source: *src/esp32s3-rtc.ads*

ESP32-S3 RTC low-power domain: retained memory, deep sleep, and wake sources.

In deep sleep the digital core (CPU + main RAM) is powered down; only the RTC domain stays alive, so on wake the chip RESETS and re-runs from the start – but data kept in RTC memory survives. This package gives you: a pointer to that retained memory, the cause of the current boot (power-on vs woken from deep sleep), and deep-sleep entry with a timer or an RTC-GPIO wake source.

No tasking is required; the operations are simple register pokes. It still lives with the embedded/full drivers (it has no certifiable-profile barrier, but is grouped with the rest of the HAL).

Public API

```
------------------------------------------------------------------------------
--  Retained memory.
--
--  RTC slow memory (8 KB at 0x5000_0000) keeps its contents across deep sleep
--  and most resets (power-on clears it).  Overlay your own data on it, e.g.
--     Boot_Count : Interfaces.Unsigned_32
--       with Import, Volatile, Address => ESP32S3.RTC.Slow_Memory;
--  (the very start may be used by the ULP coprocessor if you enable it; this
--  HAL does not, so the whole region is free here).
------------------------------------------------------------------------------

Slow_Memory      : constant System.Address := System'To_Address (16#5000_0000#);
Slow_Memory_Size : constant := 8 * 1024;

--  Word-addressed access to the retained region (a bounds-checked alternative
--  to overlaying your own variable on Slow_Memory).  Index is a 32-bit word
--  index, 0 .. 2047.
subtype Word_Index is Natural range 0 .. Slow_Memory_Size / 4 - 1;

function  Read  (Index : Word_Index) return Interfaces.Unsigned_32;
procedure Write (Index : Word_Index; Value : Interfaces.Unsigned_32);

--  Typed retained object: instantiate once per stored item, giving a distinct
--  byte Offset into the region.  The Object persists across deep sleep.
```

```
--     package Counter is new ESP32S3.RTC.Retained (Unsigned_32, Offset => 0);
--     ... Counter.Object := Counter.Object + 1; ...
generic
   type Item is private;
   Offset : Natural := 0;          --  byte offset into Slow_Memory
package Retained is
   Object : Item
     with Import, Volatile,
          Address => System.Storage_Elements."+"
                       (Slow_Memory,
                        System.Storage_Elements.Storage_Offset (Offset));
```

ESP32S3.RTC_IO

Source: src/esp32s3-rtc_io.ads

ESP32-S3 RTC-IO: the low-power GPIO domain (GPIO0 .. GPIO21 are RTC-capable).

The headline feature is pad HOLD: latch a pad at its current output level so it keeps driving while the rest of the chip changes – and, crucially, while the chip is in deep sleep (the digital core powers down, but a held RTC pad stays put) and across the reset that a deep-sleep wake causes. Use it to keep a load enabled / a reset line asserted while you sleep.

A held pad ignores ordinary GPIO writes until you Release it. No tasking is required (register pokes); RTC-IO works under every runtime profile.

Public API

```
--  The RTC-capable pads.
subtype RTC_Pin is ESP32S3.GPIO.Pin_Id range 0 .. 21;

--  Latch Pin at its current level.  After this, GPIO Set/Clear on Pin have no
--  effect on the pad until Release; the level survives deep sleep and the
--  wake reset.  (Set the pad to the wanted output level first.)
procedure Hold (Pin : RTC_Pin);

--  Release the latch; the pad follows the GPIO output register again.
procedure Release (Pin : RTC_Pin);

--  True while Pin is held.
function Is_Held (Pin : RTC_Pin) return Boolean;

--  Route Pin into the RTC domain as an input.  This connects its RTC pull (so
--  the pad holds a defined level in the RTC domain, including deep sleep) and
--  keeps it readable with ESP32S3.GPIO.Read.  Call before Set_Pull.
procedure Enable_RTC_Input (Pin : RTC_Pin);

--  RTC-domain pull-up / pull-down on a pad.  These are the RTC pulls (active
--  in the RTC domain, including deep sleep), distinct from the digital IO_MUX
--  pulls configured through ESP32S3.GPIO; they take effect once the pad is in
--  the RTC domain (Enable_RTC_Input).
type Pull_Mode is (No_Pull, Up, Down);
procedure Set_Pull (Pin : RTC_Pin; Mode : Pull_Mode);
```

ESP32S3.SD_SPI

Source: src/esp32s3-sd_spi.ads

SD / SDHC memory card over SPI (the simple, universal transport).

This layers the SD "SPI mode" command protocol (CMD0/8/58, ACMD41, CMD17/24, CRC7) on top of the task-safe ESP32S3.SPI master. The chip-select is driven as a plain GPIO so it can be held asserted across a whole command / response / data sequence (the SPI peripheral's own CS pulses per transfer, which the SD protocol cannot use).

```
Wiring (any free GPIOs -- 4 lines + power):
   card DI  (MOSI) <- ESP32 MOSI      card DO (MISO) -> ESP32 MISO
   card CLK        <- ESP32 SCLK      card CS         <- ESP32 CS (GPIO here)
   card VDD = 3V3, VSS = GND.  A 10k pull-up on DO/MISO is recommended.
```

Cards are initialised at <=400 kHz (SD spec) then run faster (Data_Clock_Hz). Addresses are 512-byte logical blocks (LBA); SDHC/SDXC use block addressing, older SDSC byte addressing – handled internally, the API is always LBA.

Task-safe: every operation takes the SPI host's Session for the whole transaction, so concurrent callers serialise. Uses finalization (via the SPI Session) -> embedded / full profiles only.

Public API

```
   --  A 512-byte logical block (the SD sector size in SPI mode).
   type Block is array (0 .. 511) of Interfaces.Unsigned_8;

   --  Logical block address (sector number).
   type Block_Address is new Interfaces.Unsigned_32;

   --  What the card turned out to be (after Initialize).
   type Card_Kind is (Unknown, SD_V1, SD_V2_SC, SD_V2_HC);
   --  SD_V1   = SDSC v1.x        SD_V2_SC = SDSC v2.0 (byte addressing)
   --  SD_V2_HC = SDHC / SDXC (block addressing)

   --  Result of an operation.
   type Status is
     (OK,
      No_Card,         --  no response to CMD0 (nothing there / not wired)
      Unusable,        --  CMD8 voltage check / OCR says not a 3V3 card
      Init_Timeout,    --  ACMD41 never reported ready
      Read_Error,      --  CMD17 / data token failure
      Write_Error);    --  CMD24 / data-response / busy failure

   --  A single card on one SPI host.  Limited (non-copyable: one object owns the
   --  card's CS line).  Holds no finalizable resource itself -- the SPI Session
   --  taken per operation does the locking -- so it needs no controlled type.
   type Card is limited private;

   ------------------------------------------------------------------------
   --  Configuration -- call once before Initialize (single-threaded).
   ------------------------------------------------------------------------

   --  Bring the SPI host up in SPI mode 0 and route the four lines, driving CS
   --  as a GPIO.  Init_Clock_Hz must be <= 400 kHz per the SD spec; Data_Clock_Hz
```

```
   --  is what Initialize switches to once the card is ready (<= 25 MHz for the
   --  default-speed SPI mode; clamp to what your wiring tolerates).
   procedure Setup (C             : out Card;
                    Host          : ESP32S3.SPI.SPI_Host;
                    Sclk, Mosi, Miso, Cs : ESP32S3.GPIO.Pin_Id;
                    Init_Clock_Hz : Positive := 400_000;
                    Data_Clock_Hz : Positive := 8_000_000);

   ------------------------------------------------------------------------
   --  Operation.
   ------------------------------------------------------------------------

   --  Run the power-up + CMD0/8/ACMD41/CMD58 handshake.  On OK the card is ready
   --  and the bus has been raised to Data_Clock_Hz; Kind reports what it is.
   procedure Initialize (C : in out Card; Result : out Status);

   --  What Initialize found (Unknown until a successful Initialize).
   function Kind (C : Card) return Card_Kind;

   --  Read / write one 512-byte block at logical address LBA.
   procedure Read_Block  (C : in out Card; LBA : Block_Address;
                          Data : out Block; Result : out Status);
   procedure Write_Block (C : in out Card; LBA : Block_Address;
                          Data : Block; Result : out Status);
```

ESP32S3.SDM

Source: ***src/esp32s3-sdm.ads***

ESP32-S3 SDM (sigma-delta modulator) – eight 1-bit density-modulated outputs.

Each channel emits a high-frequency pulse stream whose average density is set by a signed 8-bit value; pass it through an external RC low-pass filter and you get a cheap analog output (LED dimming, bias voltage, simple audio). The block lives in the GPIO sigma-delta unit (GPIO_SD).

Ownership: a channel is a shared resource handed out as a CLAIMED handle, LIMITED (non-copyable) and CONTROLLED (released automatically on scope exit). Uses finalization, so it targets the embedded/full profile.

Public API

```
   type Channel_Index is range 0 .. 7;        --  the eight SDM channels

   subtype Density_Percent is Float range 0.0 .. 100.0;

   --  Non-copyable handle to a claimed channel (check Is_Valid after Claim).
   type Channel is limited private;

   procedure Claim (C : in out Channel; Index : Channel_Index);
   function Is_Valid (C : Channel) return Boolean;
   procedure Release (C : in out Channel);

   --  Route C's output to Pin and start it at 0 % density.  Carrier_Hz is the
   --  desired pulse-stream (sigma-delta carrier) frequency: the modulator runs at
```

```
   --  the APB clock (~80 MHz) divided by an integer 1 .. 256, so the achieved rate
   --  is the nearest APB/N -- roughly 312_500 Hz .. 80_000_000 Hz.  A higher
   --  carrier is easier to smooth with an RC low-pass; the exact value rarely
   --  matters, so it is rounded to the nearest available divider.
   procedure Configure (C          : in out Channel;
                        Pin        : ESP32S3.GPIO.Pin_Id;
                        Carrier_Hz : Positive := 1_000_000);

   --  Set the average output density (0 .. 100 %).  Single register write.
   procedure Set_Density (C : Channel; Percent : Density_Percent);
```

ESP32S3.SDMMC

Source: `src/esp32s3-sdmmc.ads`

ESP32-S3 native SD/MMC host (the dedicated SDHOST controller).

Unlike ESP32S3.SD_SPI (which talks to a card over the SPI master), this drives the card on the real SD bus: a clock + bidirectional command line + 1 or 4 data lines, with the controller's own command/response state machine. It is faster than SPI mode and is how you reach an SDHC/SDXC card at speed.

Data moves in PIO/FIFO mode – the CPU pushes/pops the 512-byte block through the controller's data FIFO (BUFFIFO). No GDMA, no descriptors, hence no finalization: a library-level protected object serialises the single shared controller, so this works under EVERY runtime profile (light-tasking too).

```
Wiring -- route any free GPIOs through the GPIO matrix (4 lines for 4-bit):
   CLK (out)   CMD (bidir, pull-up)   D0..D3 (bidir, pull-ups)
   card VDD = 3V3, VSS = GND.  Pull-ups (10k-50k) on CMD and every DATA line.
There are two card slots (Slot1 / Slot2); only one card per slot.
```

Cards init at <=400 kHz then run at Data_Clock_Hz. The API is always 512-byte logical blocks (LBA); SDHC/SDXC use block addressing, SDSC byte addressing, resolved internally.

Public API

```
   --  The controller's two card slots (distinct GPIO-matrix signals + clocks).
   type Slot is (Slot1, Slot2);

   --  Data-bus width.  1-bit needs only D0; 4-bit needs D0..D3.
   type Bus_Width is (Width_1, Width_4);

   --  A 512-byte logical block.
   type Block is array (0 .. 511) of Interfaces.Unsigned_8;

   --  Logical block address (sector number).
   type Block_Address is new Interfaces.Unsigned_32;

   --  What the card turned out to be (after Initialize).
   type Card_Kind is (Unknown, SDSC, SDHC);   --  SDHC also covers SDXC

   --  Result of an operation.
   type Status is
     (OK,
      No_Card,        --  no response to CMD0/ACMD41 (nothing there / not wired)
      Unusable,       --  CMD8 / OCR says not a usable 3V3 SD card
```

```
      Init_Timeout,    --  ACMD41 never reported ready
      Cmd_Timeout,     --  a command got no response (RTO)
      Cmd_CRC,         --  response CRC error (RCRC)
      Read_Error,      --  data read failed (DCRC / DRTO / FIFO)
      Write_Error);    --  data write / programming failed

   --  A single card on one slot.  Limited (non-copyable).  Holds no finalizable
   --  resource -- the shared controller is guarded by an internal protected
   --  object -- so no controlled type is needed.
   type Card is limited private;

   ----------------------------------------------------------------------------
   --  Configuration -- call once before Initialize (single-threaded).
   ----------------------------------------------------------------------------

   --  Bring the SDHOST controller up and route this slot's lines through the
   --  GPIO matrix.  D1..D3 may be left No_Pin for a 1-bit bus.  Init_Clock_Hz
   --  must be <= 400 kHz; Data_Clock_Hz is the clock Initialize aims for.
   --
   --  If High_Speed is True and the card supports it (CMD6), Initialize switches
   --  the card into High-Speed mode and runs at up to min (Data_Clock_Hz, 50 MHz);
   --  otherwise it stays in default speed, capped at 25 MHz (the spec limit).
   --  Use Active_Clock_Hz / High_Speed_Active to see what was negotiated.
   procedure Setup (C             : out Card;
                    On            : Slot;
                    Clk, Cmd, D0  : ESP32S3.GPIO.Pin_Id;
                    D1, D2, D3    : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
                    Width         : Bus_Width := Width_1;
                    Init_Clock_Hz : Positive := 400_000;
                    Data_Clock_Hz : Positive := 20_000_000;
                    High_Speed    : Boolean  := False);

   ----------------------------------------------------------------------------
   --  Operation.
   ----------------------------------------------------------------------------

   --  Run the card-identification sequence (CMD0/8, ACMD41, CMD2/3/9/7, bus
   --  width, block length).  On OK the card is selected and the bus raised to
   --  Data_Clock_Hz; Kind reports what it is.
   procedure Initialize (C : in out Card; Result : out Status);

   --  What Initialize found (Unknown until a successful Initialize).
   function Kind (C : Card) return Card_Kind;

   ----------------------------------------------------------------------------
   --  Card identity, decoded from the CID and CSD registers Initialize read.
   ----------------------------------------------------------------------------

   --  Decoded CID (card identification) register.
   type Card_Id is record
      Manufacturer   : Interfaces.Unsigned_8 := 0;        --  MID
      OEM            : String (1 .. 2)       := "  ";     --  OID, ASCII
      Product        : String (1 .. 5)       := "     "; --  PNM, ASCII
```

```
      Revision_Major : Natural                := 0;       --  PRV high nibble
      Revision_Minor : Natural                := 0;       --  PRV low nibble
      Serial         : Interfaces.Unsigned_32 := 0;       --  PSN
      Mfg_Year       : Natural                := 0;       --  full year, e.g. 2021
      Mfg_Month      : Natural                := 0;       --  1 .. 12
```

ESP32S3.SHA

Source: src/esp32s3-sha.ads

ESP32-S3 SHA accelerator – hardware SHA-1 / SHA-224 / SHA-256 of a byte message.

The accelerator is a single shared resource, so a protected object serialises the message-load / start / read handshake, making concurrent Hash calls from different tasks safe. All three variants share the same 512-bit block and padding; they differ only in the hardware MODE and the digest length. (The block hardware also does SHA-384/512, which use a 1024-bit block and are not exposed here.) Register pokes, no finalization – works under every runtime profile.

Public API

```
   type Byte_Array is array (Natural range <>) of Interfaces.Unsigned_8;

   subtype SHA1_Digest   is Byte_Array (0 .. 19);   --  20-byte (160-bit) digest
   subtype SHA224_Digest is Byte_Array (0 .. 27);   --  28-byte (224-bit) digest
   subtype SHA256_Digest is Byte_Array (0 .. 31);   --  32-byte (256-bit) digest

   --  Hardware hashes of Data (any length, padded internally).
   function Hash_1   (Data : Byte_Array) return SHA1_Digest;
   function Hash_224 (Data : Byte_Array) return SHA224_Digest;
   function Hash_256 (Data : Byte_Array) return SHA256_Digest;
```

ESP32S3.SHT41

Source: src/esp32s3-sht41.ads

Sensirion SHT41 (SHT4x family) I2C temperature + humidity sensor driver.

Same shape as ESP32S3.PCF85063A: the driver hard-codes no board wiring – you tell Setup which host and which SDA / SCL pins the part is wired to (plus the I2C address), and the Device remembers them. Each operation then opens a short-lived I2C Session (a controlled type) for one complete exchange and lets it release the host automatically on scope exit – so the sensor shares the bus safely with the other I2C devices. No interrupt: it is read on request. Needs the controlled Session => embedded / full profiles only.

The SHT4x is command-based (no registers): a measurement is "write a 1-byte command, wait the conversion time, read 6 bytes" – two CRC-8-protected 16-bit words (temperature then humidity). Measure blocks for the conversion while holding the bus.

```
Typical use:
   declare
      Sensor : ESP32S3.SHT41.Device;
      M      : ESP32S3.SHT41.Measurement;
      St     : ESP32S3.SHT41.Status;
   begin
      ESP32S3.SHT41.Setup (Sensor, Sda => 8, Scl => 7);   --  state the wiring
```

```
   ESP32S3.SHT41.Measure (Sensor, M, St);              --  one reading
   --  M.Temperature in m°C, M.Humidity in m%RH (St = OK).
end;
```

Public API

```
--  7-bit I2C address.  The SHT41-AD1B answers at 0x44; other SHT4x variants
--  use 0x45 / 0x46.
Default_Address : constant ESP32S3.I2C.Slave_Address := 16#44#;

--  One reading, in integer milli-units (so no float library is needed):
--     Temperature 23_456  =  23.456 °C
--     Humidity    45_678  =  45.678 %RH  (clamped to 0 .. 100_000)
type Measurement is record
   Temperature : Integer := 0;    --  milli-degrees Celsius
   Humidity    : Integer := 0;    --  milli-percent relative humidity
```

ESP32S3.SPI

Source: src/esp32s3-spi.ads

ESP32-S3 general-purpose SPI master (SPI2 / SPI3), task-safe.

This is the ONLY SPI interface the application sees. The raw register driver lives in the private child ESP32S3.SPI.Engine (un-`with`-able from outside this subtree), so the unsynchronised "ZFP" primitives can't be called by accident – access is always mediated here.

Each host is guarded by a protected object; Acquire hands out a limited, non-copyable Session that owns the host exclusively (other tasks suspend on Acquire until it is released). The blocking DMA Transfer runs OUTSIDE the protected lock – the lock only arbitrates ownership. A Session releases automatically when it goes out of scope.

Requires a tasking runtime (Jorvik light-tasking or richer).

Public API

```
--  The two general-purpose hosts (SPI0/SPI1 are the flash/PSRAM controllers
--  and are deliberately not offered).
type SPI_Host is (SPI2, SPI3);

--  SPI clock polarity/phase mode (0 .. 3).
subtype SPI_Mode is Natural range 0 .. 3;

--  Sentinel for Configure_Pins: leave that line unrouted (= ESP32S3.GPIO.No_Pin).
No_Pin : constant ESP32S3.GPIO.Pad_Number := ESP32S3.GPIO.No_Pin;

--  An exclusive hold on a host.  Limited (cannot be copied -- two tasks can
--  never share one) and CONTROLLED: it releases the host automatically when
--  it goes out of scope, including during exception unwinding, so a fault
--  between Acquire and Release can't leak the lock.  Release stays available
--  to hand the host back early (it is idempotent).  This relies on
--  finalization, so these task-safe drivers target the embedded/full profile.
type Session is limited private;
```

```
   ------------------------------------------------------------------------
   --  One-time host configuration -- call once per host at startup, before any
   --  task contends for it (single-threaded).
   ------------------------------------------------------------------------

   --  Bring Host up as a full-duplex master at the given mode and bit clock
   --  (Hz, clamped to ~80 kHz .. 80 MHz) and Claim its GDMA channel.
   procedure Setup (Host     : SPI_Host;
                    Mode     : SPI_Mode := 0;
                    Clock_Hz : Positive := 1_000_000);

   --  Change just the bit clock of a Setup host (Hz, same clamp as Setup), with
   --  no GDMA re-Claim.  Lets a driver init a device slowly then run it fast
   --  (e.g. an SD card: <=400 kHz to initialise, then several MHz for data).
   procedure Set_Clock (Host : SPI_Host; Hz : Positive);

   --  Internal MOSI->MISO loopback through one GPIO pad (self-test; no wiring).
   procedure Enable_Loopback (Host : SPI_Host; Pad : ESP32S3.GPIO.Pin_Id);

   --  Route the host's signals to physical pads for an external device.  Each
   --  line is a validated GPIO pin (reserved/absent pads are caught at compile
   --  or run time); pass No_Pin to leave a line unrouted.
   procedure Configure_Pins (Host : SPI_Host;
                             Sclk : ESP32S3.GPIO.Optional_Pin;
                             Mosi : ESP32S3.GPIO.Optional_Pin;
                             Miso : ESP32S3.GPIO.Optional_Pin;
                             Cs   : ESP32S3.GPIO.Optional_Pin := No_Pin);

   ------------------------------------------------------------------------
   --  Concurrent, mutually-exclusive use.
   ------------------------------------------------------------------------

   --  Raised by Acquire if Host was never Setup -- configuration must precede
   --  ownership (see the one-time configuration section above).
   Not_Initialized : exception;

   --  Raised by Transfer if its Session does not currently hold a host.  The
   --  transfer reaches the hardware only through one ownership-checked gateway
   --  in the body, so "transfer without holding the host" fails loudly.
   Not_Owned : exception;

   --  Take exclusive ownership of a Setup host.  Suspends until no other task
   --  holds it.  Keep it across a whole transaction, then Release / let it go
   --  out of scope.  Raises Not_Initialized if Host was never Setup.
   procedure Acquire (S : in out Session; Host : SPI_Host);

   --  Full-duplex DMA transfer of Length bytes (1 .. 4095) on the held host:
   --  shift Tx out on MOSI, capture MISO into Rx.  Blocking.  Buffers in
   --  internal SRAM.  Raises Not_Owned unless S currently holds a host.
   procedure Transfer (S : Session; Tx, Rx : System.Address; Length : Natural);

   --  Relinquish ownership (lets a waiting task proceed).  Harmless if already
   --  released.  Always release a Session you Acquired.
   procedure Release (S : in out Session);
```

ESP32S3.SPI.Engine *(internal)*

Source: src/esp32s3-spi-engine.ads

RAW SPI2/SPI3 register driver – the ZFP-safe *mechanism* with NO mutual exclusion. PRIVATE child: only the ESP32S3.SPI subtree may use it; the application cannot `with` it, and reaches SPI only through the task-safe parent (ESP32S3.SPI). See the parent for the design rationale.

(SPI_Host / SPI_Mode / No_Pin are declared in the parent and used here by child visibility.)

Public API

```
   --  A configured host plus its Claimed GDMA channel.  Limited because it holds
   --  a (limited, controlled) GDMA Channel; built in place by Open.
   type Bus is limited private;

   procedure Open (B        : in out Bus;
                   Host     : SPI_Host;
                   Mode     : SPI_Mode;
                   Clock_Hz : Positive);

   function Is_Open (B : Bus) return Boolean;

   --  Change just the bit clock of an already-Open bus (no GDMA re-Claim).
   procedure Set_Clock (B : Bus; Hz : Positive);

   procedure Configure_Pins (B : Bus;
                             Sclk : ESP32S3.GPIO.Optional_Pin;
                             Mosi : ESP32S3.GPIO.Optional_Pin;
                             Miso : ESP32S3.GPIO.Optional_Pin;
                             Cs   : ESP32S3.GPIO.Optional_Pin := No_Pin);

   procedure Enable_Loopback (B : Bus; Pad : ESP32S3.GPIO.Pin_Id);

   procedure Transfer (B : Bus; Tx, Rx : System.Address; Length : Natural);

   procedure Close (B : in out Bus);
```

ESP32S3.ST7789

Source: src/esp32s3-st7789.ads

ST7789 (ST77xx-family) SPI display controller driver.

4-wire SPI, write-only (CLK + MOSI; no MISO – the controller cannot be read back, so there is no probe / status). Three GPIOs the driver drives directly: DC (data/command), CS (chip select), and an optional RST (reset; if No_Pin a software reset is used instead). Pixels are 16-bit RGB565, MSB-first.

```
Locking -- TWO levels, like the TCA9555:
  * A Session is an exclusive, RAII hold on ONE display, acquired like the
    RTC.  Hold it across a whole sequence of operations so no other task can
    corrupt the controller mid-sequence.
  * The SPI host is locked only INSIDE each operation (assert CS, transfer,
```

```
    deassert CS, release), so the SPI peripheral is used "only as long as
    necessary" and is free between operations for another task / device.  CS
    is high whenever the bus is released, so nothing interferes.
The per-display guards are a fixed library-level array keyed by the CS pin (a
GPIO uniquely identifies one display), so no protected object lives in a
Device.  Uses controlled Sessions => embedded / full profiles only.

Typical use:
   declare
      LCD : ESP32S3.ST7789.Device;
      S   : ESP32S3.ST7789.Session;
   begin
      ESP32S3.ST7789.Setup (LCD, Sclk => 12, Mosi => 13, DC => 16, CS => 10,
                            Width => 240, Height => 240);
      ESP32S3.ST7789.Acquire (S, LCD);          --  protect this display
      ESP32S3.ST7789.Init (S);
      ESP32S3.ST7789.Fill (S, ESP32S3.ST7789.Blue);
   end;                                          --  Session auto-released
```

Public API

```
   --  16-bit RGB565 colour (5 red, 6 green, 5 blue), MSB-first on the wire.
   type Color is mod 2 ** 16;
   function RGB (R, G, B : Natural) return Color;   --  each 0 .. 255

   Black : constant Color := 16#0000#;
   White : constant Color := 16#FFFF#;
   Red   : constant Color := 16#F800#;
   Green : constant Color := 16#07E0#;
   Blue  : constant Color := 16#001F#;

   --  A row-major block of pixels for Draw_Bitmap.
   type Color_Array is array (Natural range <>) of Color;

   type Rotation is (Rot_0, Rot_90, Rot_180, Rot_270);

   type Device  is limited private;
   type Session is limited private;

   --  Raised by Acquire if the Device was never Setup, and by an operation whose
   --  Session does not currently hold a display.
   Not_Initialized : exception;
   Not_Owned       : exception;

   ------------------------------------------------------------------------------
   --  One-time configuration -- call once per display at startup.
   ------------------------------------------------------------------------------

   --  Record the wiring + geometry and bring the SPI host up (mode 0, full-duplex
   --  master; CS / MISO are NOT routed to the peripheral -- CS is driven here as
   --  a GPIO).  Width/Height are the panel resolution; X_Offset/Y_Offset are the
   --  controller-to-panel origin offset some panels need.  No pin defaults for
   --  the four routed lines.
   procedure Setup
```

```
   (Dev               : out Device;
    Sclk, Mosi, DC, CS : ESP32S3.GPIO.Pin_Id;
    Width             : Positive                    := 240;
    Height            : Positive                    := 240;
    RST               : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
    X_Offset, Y_Offset : Natural                    := 0;
    Host              : ESP32S3.SPI.SPI_Host        := ESP32S3.SPI.SPI2;
    Mode              : ESP32S3.SPI.SPI_Mode        := 0;
    Clock_Hz          : Positive                    := 40_000_000);

--  Take / release exclusive ownership of the display.

procedure Acquire (S : in out Session; Dev : Device);
procedure Release (S : in out Session);

--  Operations -- each takes the held Session and locks the SPI host only for
--  its own transfers.  Raise Not_Owned unless S currently holds a display.

--  Reset (hardware via RST if wired, else software) and run the power-on init
--  sequence (sleep-out, 16-bit colour, normal display, display on).  (Named
--  Init, not Initialize, to avoid clashing with the controlled type's own.)
procedure Init (S : Session);

procedure Display_On  (S : Session);
procedure Display_Off (S : Session);
procedure Set_Rotation (S : Session; Rot : Rotation);  --  sets MADCTL
procedure Invert       (S : Session; On : Boolean);    --  colour inversion
procedure Sleep        (S : Session; On : Boolean);

--  Fill the whole screen / a rectangle with one colour (clipped to the panel).
procedure Fill      (S : Session; C : Color);
procedure Fill_Rect (S : Session; X, Y, W, H : Natural; C : Color);
procedure Set_Pixel (S : Session; X, Y : Natural; C : Color);

--  Blit a W x H block of pixels (row-major) at (X, Y).  Pixels'Length must be
--  W * H.
procedure Draw_Bitmap
  (S : Session; X, Y, W, H : Natural; Pixels : Color_Array);
```

ESP32S3.ST7789.Fonts

Source: *src/esp32s3-st7789-fonts.ads*

Proportional anti-aliased / monochrome text for the ST7789, an instantiation of the panel-agnostic ESP32S3.Fonts.Render engine bound to this driver's pixel type and Draw_Bitmap. Hold the display Session (the two-level lock) and paint the text background first, then:

```
ESP32S3.ST7789.Fonts.Draw_Text
  (S, My_Font, X => 6, Baseline => 20, Str => "Hello",
   FG => (255, 255, 255), BG => (0, 0, 0));
```

Font values come from generated atlases (see tools/gen_font.py); they are ESP32S3.Fonts.Font and so are shared unchanged with any other panel's instantiation. Distinct from ESP32S3.ST7789.Text, which is the built-in 5x7 bitmap font.

Public API

```
(Color       => ESP32S3.ST7789.Color,
 Color_Array => ESP32S3.ST7789.Color_Array,
 Surface     => ESP32S3.ST7789.Session,
 To_Color    => ESP32S3.ST7789.RGB,
 Blit        => ESP32S3.ST7789.Draw_Bitmap);
```

ESP32S3.ST7789.Text

Source: src/esp32s3-st7789-text.ads

5x7 bitmap text on top of ESP32S3.ST7789.

Built purely on the parent's Draw_Bitmap primitive: each glyph is a 5x7 bitmap rendered into a 6x8 cell (one column + one row of inter-character spacing) and blitted as one windowed write. Cells are OPAQUE – a glyph paints both foreground and background – because the panel is write-only (no framebuffer to read back for a transparent overlay).

Takes the same held Session as the rest of the driver, so text shares the two-level locking: the display stays owned across a sequence of Draw_Text calls while each one locks the SPI host only for its own transfers.

```
ESP32S3.ST7789.Text.Draw_Text
  (S, X => 8, Y => 8, Str => "Hello", FG => White, BG => Black);
```

Public API

```
--  Cell geometry at Scale = 1: a 5x7 glyph in a 6x8 cell.  At Scale = N each
--  font pixel becomes an N x N block, so a character occupies
--  Cell_Width * N by Cell_Height * N pixels.
Cell_Width  : constant := 6;
Cell_Height : constant := 8;

--  Draw one character with its cell's top-left at (X, Y).  FG paints the set
--  pixels, BG the rest (including the spacing column/row).  Characters
--  outside printable ASCII (0x20 .. 0x7E) render as a blank cell.
procedure Draw_Char
  (S      : Session;
   X, Y   : Natural;
   Ch     : Character;
   FG, BG : Color;
   Scale  : Positive := 1);

--  Draw a string with its first cell at (X, Y), advancing X by
--  Cell_Width * Scale per character.  An ASCII LF (Character'Val (10))
--  returns to the start column and drops down one line (Cell_Height * Scale).
--  No automatic right-edge wrap -- glyphs that would start past the panel are
--  skipped by the driver's bounds check.
procedure Draw_Text
  (S      : Session;
```

```
      X, Y   : Natural;
      Str    : String;
      FG, BG : Color;
      Scale  : Positive := 1);
```

ESP32S3.TCA9555

Source: src/esp32s3-tca9555.ads

Texas Instruments TCA9555 16-bit I2C GPIO expander driver.

Two 8-bit ports (P0 = pins 0..7, P1 = pins 8..15). The 7-bit address is 0x20 + the A2/A1/A0 strap value, so up to 8 parts share one bus.

```
Locking -- TWO levels, which is the point of this driver:
  * A Session is an exclusive, RAII hold on ONE expander (acquire it like the
    RTC's host).  Hold it across as many operations as you like; it keeps
    other tasks off THAT chip (so a per-pin read-modify-write is safe) while
    it is held.  Two Sessions for the same physical chip (same host+address)
    serialise; different chips do not.
  * The I2C HOST is locked only INSIDE each read / write, then released -- so
    while you hold an expander's Session the bus is free between your
    transactions, and another task can drive a different chip (or the RTC /
    IMU / SHT41) in the gaps.
```

The per-device locks are a fixed library-level array keyed by (host, strap value) – the same shape as ESP32S3.I2C's per-host guards – so no protected object lives in a Device (which would be a forbidden local PO).

Uses controlled Sessions (finalization) => embedded / full profiles only.

```
Typical use:
   declare
      Exp : ESP32S3.TCA9555.Device;
      S   : ESP32S3.TCA9555.Session;
      St  : ESP32S3.TCA9555.Status;
      V   : ESP32S3.TCA9555.Port_Value;
   begin
      ESP32S3.TCA9555.Setup (Exp, Addr => 0, Sda => 8, Scl => 7);
      ESP32S3.TCA9555.Acquire (S, Exp);                  --  protect this chip
      ESP32S3.TCA9555.Set_Directions (S, Inputs => 16#FF00#, Result => St);
      ESP32S3.TCA9555.Write_Port (S, 16#00A5#, St);      --  bus free between
      ESP32S3.TCA9555.Read_Port  (S, V, St);             --  these two
   end;                                                  --  Session auto-released
```

Public API

```
   --  Base address and the A2/A1/A0 strap value (0 .. 7) -> 0x20 .. 0x27.
   Base_Address : constant ESP32S3.I2C.Slave_Address := 16#20#;
   subtype Hardware_Address is Natural range 0 .. 7;

   --  16 I/O pins: 0 .. 7 = Port 0, 8 .. 15 = Port 1.
   type Pin_Number is range 0 .. 15;

   --  A whole-expander value: bit i corresponds to Pin_Number i.
```

```
type Port_Value is mod 2 ** 16;

type Direction is (Output, Input);
type Pin_State is (Low, High);

--  Result of a bus operation.  Bus_Error: the expander did not ACK.
type Status is (OK, Bus_Error);

--  A configured expander (host + address).  Holds no lock itself.
type Device is limited private;

--  An exclusive, RAII hold on one expander.  Limited + controlled: releases
--  the device on scope exit (including on exception), like the RTC Session.
type Session is limited private;

--  Raised by Acquire if the Device was never Setup.
Not_Initialized : exception;
--  Raised by an operation whose Session does not currently hold a device.
Not_Owned : exception;

--------------------------------------------------------------------------
--  One-time configuration -- call once per device at startup.
--------------------------------------------------------------------------

--  Record the wiring + strap address (Addr 0..7 -> 0x20+Addr) and bring the
--  I2C host up.  Int_Pin is the GPIO the active-low INT output is wired to, or
--  No_Pin (this board: not connected); arming it is the job of the
--  ESP32S3.TCA9555.Interrupts child.  No pin defaults for Sda/Scl.
procedure Setup
  (Dev      : out Device;
   Addr     : Hardware_Address;
   Sda      : ESP32S3.GPIO.Pin_Id;
   Scl      : ESP32S3.GPIO.Pin_Id;
   Int_Pin  : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
   Host     : ESP32S3.I2C.I2C_Host      := ESP32S3.I2C.I2C0;
   Clock_Hz : Positive                  := 400_000);

--  The INT pin Dev was set up with (No_Pin if none).
function Interrupt_Pin (Dev : Device) return ESP32S3.GPIO.Optional_Pin;

--------------------------------------------------------------------------
--  Take / release exclusive ownership of the expander.
--------------------------------------------------------------------------

--  Suspend until no other Session holds this device, then own it.  Raises
--  Not_Initialized if Dev was never Setup.
procedure Acquire (S : in out Session; Dev : Device);

--  Hand the device back early (idempotent; also happens on scope exit).
procedure Release (S : in out Session);

--------------------------------------------------------------------------
--  Operations -- each takes the held Session and locks the I2C host only for
```

```
   --  its own transaction(s).  Raise Not_Owned unless S currently holds a device.
   ---------------------------------------------------------------------------

   --  Pin direction.  In Inputs, a 1 bit makes that pin an input (the chip's
   --  power-on default), a 0 bit an output.  One transaction.
   procedure Set_Directions
     (S : Session; Inputs : Port_Value; Result : out Status);
   procedure Set_Direction
     (S : Session; Pin : Pin_Number; Dir : Direction; Result : out Status);

   --  Drive the output register.  Write_Pin is a read-modify-write (two bus
   --  transactions, safe because the Session is held).
   procedure Write_Port (S : Session; Value : Port_Value; Result : out Status);
   procedure Write_Pin
     (S : Session; Pin : Pin_Number; State : Pin_State; Result : out Status);

   --  Read back the direction (config) and output registers (what was last set).
   procedure Read_Directions
     (S : Session; Inputs : out Port_Value; Result : out Status);
   procedure Read_Outputs
     (S : Session; Value : out Port_Value; Result : out Status);
   procedure Read_Polarity
     (S : Session; Inverted : out Port_Value; Result : out Status);

   --  Sample the input register (the actual pin levels, including driven outputs).
   procedure Read_Port (S : Session; Value : out Port_Value; Result : out Status);
   procedure Read_Pin
     (S : Session; Pin : Pin_Number; State : out Pin_State; Result : out Status);

   --  Input polarity inversion: a 1 bit inverts that input's reported level.
   procedure Set_Polarity
     (S : Session; Inverted : Port_Value; Result : out Status);
```

ESP32S3.TCA9555.Interrupts

Source: src/esp32s3-tca9555-interrupts.ads

The TCA9555 INT line, on the GPIO the device was Setup with.

INT is active-low and open-drain: it idles high (via a pull-up) and the chip pulls it low when an input pin changes, holding it low until the input register is read. So the pin is configured as an input with the internal pull-up on, triggering on the FALLING edge.

These act on the INT pin stored in the Device by Setup – pass No_Pin there (this board: INT not connected) and Attach / Detach are no-ops. UNTESTED on this board, since the INT line is not wired.

The Action runs in interrupt context (see ESP32S3.GPIO.Interrupts): keep it short – set a Suspension_Object or bump an Atomic flag, then read the inputs (which also clears INT) in a normal task.

Public API

```
   subtype Callback is ESP32S3.GPIO.Interrupts.Callback;

   --  Configure Dev's INT pin as a pulled-up input and deliver a falling-edge
```

```
   --  interrupt to Action.  No-op if Dev was set up with No_Pin.
   procedure Attach (Dev : Device; Action : Callback);

   --  Stop delivering Dev's INT interrupt.  No-op if Dev has no INT pin.
   procedure Detach (Dev : Device);
```

ESP32S3.Temperature

Source: *src/esp32s3-temperature.ads*

ESP32-S3 on-chip temperature sensor.

Reports the die temperature of the SoC (NOT ambient air – the chip self-heats under load, so an idle board typically reads a few degrees above room temperature). Accuracy is roughly +/- 1 C after the part's factory trim, best near the middle of the selected measurement range.

Bring-up (done by Initialize) follows esp-idf's temperature_sensor_ll: gate the SAR peripheral clock, pulse-reset it, open the analog REGI2C bus, program the sensor's DAC range over that bus (a ROM call), then power the sensor up. Each reading drives the dump-out / ready handshake that latches a fresh conversion before sampling SAR_TSENS_OUT.

Task-safe: the sensor is owned by a protected object, so concurrent Read_* from different tasks are serialised automatically (Read_* busy-wait microseconds for the conversion under that lock). Initialize is optional – the first Read brings the sensor up with the default range if you skip it. (Holding a protected object, this package requires a tasking runtime.)

Public API

```
   --  Hardware measurement ranges (TRM "temperature_sensor_attributes").  The
   --  sensor is most accurate near the middle of the chosen range; pick the one
   --  that brackets your expected die temperature.  The default suits a typical
   --  board running near room temperature.
   type Measure_Range is
     (Range_Minus10_80,    --  -10 .. 80 C   (default)
      Range_20_100,        --   20 .. 100 C
      Range_50_125,        --   50 .. 125 C
      Range_Minus30_50,    --  -30 .. 50 C
      Range_Minus40_20);   --  -40 .. 20 C

   --  Power up and configure the sensor.  Call once before any Read_*; safe to
   --  call again to switch range.  (Read_* auto-initialise with the default
   --  range if you never call this.)
   procedure Initialize (Span : Measure_Range := Range_Minus10_80);

   --  Latest die temperature in centi-degrees Celsius, signed
   --  (e.g. 1807 = 18.07 C).  Triggers a fresh conversion and busy-waits
   --  (microseconds) for it.
   function Read_Centi_Celsius return Integer;

   --  Latest die temperature in whole degrees Celsius, truncated toward zero.
   function Read_Celsius return Integer;

   --  Raw 8-bit sensor code (0 .. 255), if you'd rather apply your own curve.
   function Read_Raw return ESP32S3_Registers.Byte;
```

ESP32S3.Timer

Source: *src/esp32s3-timer.ads*

ESP32-S3 general-purpose timers (timer groups TIMG0 / TIMG1).

Each group has one 54-bit up/down counter clocked from the APB clock through a 16-bit prescaler, with a programmable alarm. This driver exposes the two general-purpose timers (the per-group watchdogs are separate and untouched).

Ownership: a timer is a shared resource handed out as a CLAIMED handle, LIMITED (non-copyable) and CONTROLLED (released automatically on scope exit). Uses finalization, so it targets the embedded/full profile.

Public API

```
type Timer_Index is range 0 .. 1;          --  0 = TIMG0, 1 = TIMG1

type Ticks is new Interfaces.Unsigned_64;

--  Non-copyable handle to a claimed timer (check Is_Valid after Claim).
type Timer is limited private;

procedure Claim (T : in out Timer; Index : Timer_Index);
function Is_Valid (T : Timer) return Boolean;
procedure Release (T : in out Timer);

--  Configure T to count up at Tick_Hz ticks/second (default 1 MHz = 1 µs per
--  tick; max ~80 MHz / 1, min ~80 MHz / 65536).  The counter is stopped and
--  reset to 0; call Start.
procedure Configure (T : in out Timer; Tick_Hz : Positive := 1_000_000);

procedure Start (T : Timer);
procedure Stop  (T : Timer);

--  Reload the counter to 0 (whether running or stopped).
procedure Reset (T : Timer);

--  Current counter value (latched then read).
function Value (T : Timer) return Ticks;

--  Raise the alarm interrupt-status flag when the counter reaches At_Ticks.
procedure Set_Alarm (T : Timer; At_Ticks : Ticks);

--  True once the alarm has fired (the flag stays set until Clear_Alarm).
function Alarm_Fired (T : Timer) return Boolean;

procedure Clear_Alarm (T : Timer);
```

ESP32S3.Touch

Source: *src/esp32s3-touch.ads*

ESP32-S3 capacitive touch sensor (v2): 14 channels on GPIO1 .. GPIO14.

Each channel measures the self-capacitance of its pad by counting charge/discharge cycles in a fixed window; a finger near the pad raises the capacitance and changes the count. The FSM scans the enabled channels continuously on the RTC timer; Read returns the latest raw count.

No tasking is required (register pokes); it lives in the RTC/SENS domain.

Public API

```
   --  Touch channel n is wired to GPIO n.
   type Channel is range 1 .. 14;

   --  The GPIO a channel uses.
   function Pad (Ch : Channel) return ESP32S3.GPIO.Pin_Id;

   --  Bring the touch controller up and start the scanning FSM.  Call once.
   procedure Setup;

   --  Route channel Ch's pad into touch mode and add it to the scan.
   procedure Enable (Ch : Channel);

   --  Latest raw capacitance count for Ch (0 .. ~4 million; higher = more
   --  capacitance).  A bare pad reads a stable non-zero baseline; a touch raises
   --  it.  0 means the channel has not produced a measurement yet.
   function Read (Ch : Channel) return Natural;

   --  Touch detection: True when Ch's current count deviates from Reference by
   --  more than Margin (either direction).  Capture Reference with Read while the
   --  pad is untouched; a finger then moves the count past the margin.  (This is
   --  a software comparison on the live Read value -- simple and deterministic.)
   function Touched (Ch        : Channel;
                     Reference : Natural;
                     Margin    : Natural := 20_000) return Boolean;
```

ESP32S3.TWAI

Source: *src/esp32s3-twai.ads*

ESP32-S3 TWAI (Two-Wire Automotive Interface = CAN 2.0), task-safe.

One CAN controller (SJA1000-compatible). This driver sends and receives both standard (11-bit identifier) and extended (29-bit, CAN 2.0B) data frames. A real bus needs an external transceiver on the TX/RX pins; for a wiring-free self-test the controller's self-test mode lets it transmit and receive its own frame (no second node to acknowledge), with TX looped back to RX through one GPIO pad.

The single controller is guarded by a protected object; Acquire hands out a limited, controlled Session that owns it exclusively and releases on scope exit. Uses finalization, so it targets the embedded/full profile.

Public API

```
   --  Bus mode: Normal drives a real bus; Listen_Only never transmits/acks;
   --  Self_Test transmits + self-receives without an external acknowledgement.
   type Bus_Mode is (Normal, Listen_Only, Self_Test);

   subtype Data_Length is Natural range 0 .. 8;
```

```
   type Data_Bytes is array (0 .. 7) of Interfaces.Unsigned_8;

   --  CAN identifiers come in two widths.  Each frame type carries its own, so
   --  the identifier is range-checked to the standard it belongs to, and Send is
   --  overloaded on it --- you cannot put a 29-bit id in a standard frame.
   subtype Standard_Id is Interfaces.Unsigned_32 range 0 .. 16#7FF#;        --  11-bit
   subtype Extended_Id is Interfaces.Unsigned_32 range 0 .. 16#1FFF_FFFF#;  --  29-bit

   --  A standard (11-bit ID) CAN frame.  Remote => True makes it a remote-transmission
   --  request (RTR): it carries Id and Length (the requested data length) but no
   --  Data on the wire -- a node owning that Id is expected to answer with a data
   --  frame.  Remote => False (the default) is an ordinary data frame.
   type Standard_Frame is record
      Id     : Standard_Id := 0;
      Remote : Boolean     := False;
      Length : Data_Length := 0;
      Data   : Data_Bytes  := (others => 0);
```

ESP32S3.TWAI.Engine *(internal)*

Source: src/esp32s3-twai-engine.ads

RAW TWAI (CAN) register driver – the ZFP-safe *mechanism* with NO mutual exclusion. PRIVATE child: only the ESP32S3.TWAI subtree may use it; the application reaches TWAI only through the task-safe parent, which hides the Bus handle and hands it out solely through its ownership-checked gateway. This is the only unit that names the TWAI controller registers.

(Frame / Data_Length / Data_Bytes / Bus_Mode are declared in the parent and used here by child visibility.)

Public API

```
   --  A configured controller.  (The TWAI block is a singleton, so this carries
   --  only the operating-mode flag + a validity bit -- the registers live at a
   --  fixed address reached in the body.)
   type Bus is private;

   --  Bring the controller up at (about) Bit_Rate bit/s in the given mode,
   --  accepting all identifiers.
   function Open (Mode : Bus_Mode; Bit_Rate : Positive) return Bus;

   --  Route TWAI TX/RX to physical pads (for a real transceiver).
   procedure Configure_Pins (B  : Bus;
                             Tx : ESP32S3.GPIO.Optional_Pin;
                             Rx : ESP32S3.GPIO.Optional_Pin);

   --  Loop TX back to RX through one pad (wiring-free self-test).
   procedure Enable_Loopback (B : Bus; Pad : ESP32S3.GPIO.Pin_Id);

   --  Transmit a frame and block until the controller finishes (self-test
   --  self-RX).  Extended selects the 29-bit on-wire format; Remote sends an RTR
   --  request (Length only, no Data on the wire); Id carries 11 or 29 significant
   --  bits accordingly.
```

```
procedure Send (B : Bus; Extended, Remote : Boolean;
                Id : Interfaces.Unsigned_32;
                Length : Data_Length; Data : Data_Bytes);

--  Is a received frame waiting (RX buffer non-empty)?  RX_Extended reports the
--  waiting frame's width (the frame-info FF bit, valid only when RX_Pending).
function RX_Pending  (B : Bus) return Boolean;
function RX_Extended (B : Bus) return Boolean;

--  Read the waiting frame if its width matches Want_Extended: decode Id/Remote/
--  Length/Data, release the buffer, Got => True.  (An RTR frame has no Data.)
--  Got => False (buffer left intact) if the waiting frame is the other width,
--  or none arrived within a short timeout.
procedure Receive (B : Bus; Want_Extended : Boolean;
                   Id : out Interfaces.Unsigned_32; Remote : out Boolean;
                   Length : out Data_Length; Data : out Data_Bytes;
                   Got : out Boolean);
```

ESP32S3.TX1812

Source: *src/esp32s3-tx1812.ads*

ESP32-S3 driver for TX1812 addressable RGB LEDs.

The TX1812 (like the WS2812 / "NeoPixel" family) is a single-wire, daisy- chainable RGB LED: 24 bits of colour are clocked in MSB-first as a precisely timed pulse train (a '1' is a long-high/short-low bit, a '0' short-high/ long-low), and a >80 us low "reset" latches them. This driver generates that waveform with the RMT peripheral (one RMT symbol per data bit).

Ownership follows the HAL idiom: a Strip is a CLAIMED handle that takes an RMT transmit channel on Acquire and releases it automatically on scope exit (its RMT channel component is controlled). Acquire the Strip, Set pixel colours into its buffer, then Show to clock them out.

LIMITATION (single LED for now): the underlying RMT.Transmit sends at most 47 symbols per burst, and each LED needs 24 – so today a Strip drives ONE LED (Count => 1). Driving a longer string needs RMT wrap/refill support, which is a later step; the API is already shaped for it (Count, per-pixel Set), so only Show's transport changes.

Public API

```
--  A pixel colour (8 bits per channel; the wire order is handled internally).
type Color is record
   R, G, B : Unsigned_8 := 0;
```

ESP32S3.UART

Source: *src/esp32s3-uart.ads*

ESP32-S3 UART (UART0 / UART1 / UART2), task-safe.

This is the ONLY UART interface the application sees. The raw register driver lives in the private child ESP32S3.UART.Engine (un-`with`-able from outside this subtree), so the unsynchronised primitives can't be called by accident – access is always mediated here.

Each port is guarded by a protected object; Acquire hands out a limited, non-copyable Session that owns the port exclusively (other tasks suspend on Acquire until it is released). The blocking Write / Read run OUTSIDE the protected lock – the lock only arbitrates ownership.

Like the other HAL drivers this targets the embedded profile (full exceptions); see the library README. Requires a tasking runtime.

Public API

```
   --  The three general-purpose UART controllers.  (UART0 is the ROM console on
   --  dev boards, but this bare runtime uses USB-Serial-JTAG for the console, so
   --  UART0's pads are free to repurpose.)
   type UART_Port is (UART0, UART1, UART2);

   subtype Baud_Rate is Positive range 300 .. 5_000_000;
   type Data_Bits   is range 5 .. 8;
   type Parity_Mode is (None, Even, Odd);
   type Stop_Bits   is (One, Two);

   type Byte is new Interfaces.Unsigned_8;
   type Byte_Array is array (Natural range <>) of Byte;

   --  An exclusive hold on a port.  Limited (cannot be copied) and CONTROLLED:
   --  releases the port automatically on scope exit, including during exception
   --  unwinding, so a fault between Acquire and Release can't leak the lock.
   --  Release stays available to hand the port back early (idempotent).  This
   --  relies on finalization -> these task-safe drivers target embedded/full.
   type Session is limited private;

   ------------------------------------------------------------------------
   --  One-time port configuration -- call once per port at startup, before any
   --  task contends for it (single-threaded).  This is the ONLY port-based
   --  configuration call: everything that changes a port AFTER Setup is done
   --  through a held Session (below), so a reconfiguration can never race
   --  another task's transaction on the same port.
   ------------------------------------------------------------------------

   --  Bring Port up at the given baud and frame format (default 115200 8-N-1)
   --  AND route its lines to physical pads.  The four pins are all optional
   --  (default No_Pin = unrouted), so a link routes only what it uses:
   --     Setup (UART1, Tx => 17, Rx => 16);               --  full-duplex link
   --     Setup (UART1, Rx => 18);                         --  RX only (e.g. GPS)
   --     Setup (UART1, Tx => 17, Rx => 16,
   --                   Rts => 19, Cts => 20);             --  + RTS/CTS flow ctl
   --     Setup (UART1);                                   --  no pins (loopback)
   --
   --  Giving Rts enables RX flow control: the controller drives RTS to pause the
   --  peer once our RX FIFO reaches Rx_Flow_Threshold bytes (of 128).  Giving Cts
   --  enables TX flow control: the transmitter only sends while the peer asserts
   --  CTS.  Inputs (RX, CTS) get an internal pull-up so an idle line reads high.
   --  (Re-route pins, change baud/format, or flip polarity later through a held
   --  Session -- see the concurrent section below.)
   procedure Setup (Port   : UART_Port;
                    Baud   : Baud_Rate   := 115_200;
```

```
                  Bits   : Data_Bits   := 8;
                  Parity : Parity_Mode := None;
                  Stop   : Stop_Bits   := One;
                  Tx     : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
                  Rx     : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
                  Rts    : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
                  Cts    : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
                  Rx_Flow_Threshold : Natural := 100);

   ---------------------------------------------------------------------------
   --  Concurrent, mutually-exclusive use.  Acquire a Session, then run every
   --  transfer AND every post-Setup reconfiguration through it -- so changing a
   --  parameter requires owning the port, and can never race another task.
   ---------------------------------------------------------------------------

   --  Raised by Acquire if Port was never Setup -- configuration must precede
   --  ownership (see the one-time configuration section above).
   Not_Initialized : exception;

   --  Raised by ANY operation below (transfer or reconfiguration) if its
   --  Session does not currently hold a port.  Every such call reaches the
   --  hardware only through one ownership-checked gateway in the body, so
   --  "use a port without holding it" fails loudly rather than silently.
   Not_Owned : exception;

   --  Take exclusive ownership of a Setup port (suspends until it is free).
   --  Raises Not_Initialized if Port was never Setup.
   procedure Acquire (S : in out Session; Port : UART_Port);

   --  Push Data to the TX FIFO, waiting for room.  Returns once every byte is
   --  queued (not necessarily fully shifted out).  Raises Not_Owned unless S
   --  currently holds a port.
   procedure Write (S : Session; Data : Byte_Array);

   --  Read up to Data'Length bytes into Data, waiting briefly for each; Count is
   --  how many were actually received (short read on timeout).  Raises Not_Owned
   --  unless S currently holds a port.
   procedure Read (S : Session; Data : out Byte_Array; Count : out Natural);

   --  Bytes currently waiting in the RX FIFO.  Raises Not_Owned unless S holds
   --  a port.
   function Available (S : Session) return Natural;

   --  ----  Post-Setup reconfiguration -- all require the held port  ----------
   --  Each acts on the port S currently holds and raises Not_Owned unless S is
   --  active (the held Session is the proof that no other task can be mid-
   --  transfer).  Like the transfers above, they reach the hardware only through
   --  the body's single ownership-checked gateway.

   --  Re-program one frame attribute independently, leaving the others (and the
   --  pin routing) untouched.  Each is a read-modify-write of just that
   --  attribute and takes effect immediately.
   procedure Set_Baud      (S : Session; Baud   : Baud_Rate);
```

```
   procedure Set_Data_Bits (S : Session; Bits   : Data_Bits);
   procedure Set_Parity    (S : Session; Parity : Parity_Mode);
   procedure Set_Stop_Bits (S : Session; Stop   : Stop_Bits);

   --  Re-route the held port's lines to physical pads (same semantics as Setup's
   --  pin parameters; sets the FULL flow-control + inversion state, so a
   --  reconfigure without a line also turns that line/flow off).  ALL optional.
   --  The *_Invert flags set each line's polarity (see Set_Inversion); default
   --  False = active-high / standard idle-high UART.
   procedure Configure_Pins
     (S    : Session;
      Tx   : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
      Rx   : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
      Rts  : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
      Cts  : ESP32S3.GPIO.Optional_Pin := ESP32S3.GPIO.No_Pin;
      Rx_Flow_Threshold : Natural := 100;
      Tx_Invert  : Boolean := False;
      Rx_Invert  : Boolean := False;
      Rts_Invert : Boolean := False;
      Cts_Invert : Boolean := False);

   --  Independently invert (or un-invert) each line's polarity on the held port.
   --  Sets the full state of all four lines; default False clears inversion.
   procedure Set_Inversion
     (S    : Session;
      Tx   : Boolean := False;
      Rx   : Boolean := False;
      Rts  : Boolean := False;
      Cts  : Boolean := False);

   --  Controller-level internal TX->RX loopback on the held port (a self-test
   --  that needs NO pins and no wiring -- UART is push-pull, so unlike I2C this
   --  fully works on-chip): bytes written come straight back on Read.
   procedure Enable_Loopback (S : Session; On : Boolean := True);

   --  Relinquish ownership (lets a waiting task proceed).  Harmless if already
   --  released.  Always release a Session you Acquired.
   procedure Release (S : in out Session);
```

ESP32S3.UART.Engine *(internal)*

Source: *src/esp32s3-uart-engine.ads*

RAW UART0/1/2 register driver – the ZFP-safe *mechanism* with NO mutual exclusion. PRIVATE child: only the ESP32S3.UART subtree may use it; the application reaches UART only through the task-safe parent. See the parent for the design rationale.

(UART_Port / Baud_Rate / Data_Bits / Parity_Mode / Stop_Bits / Byte / Byte_Array are declared in the parent and used here by child visibility.)

Public API

```
   --  A configured UART port.
   type Bus is private;

   function Open (Port   : UART_Port;
                  Baud   : Baud_Rate;
                  Bits   : Data_Bits;
                  Parity : Parity_Mode;
                  Stop   : Stop_Bits) return Bus;

   function Is_Open (B : Bus) return Boolean;

   --  Route TXD (push-pull out) / RXD (in, pulled up) and, optionally, the
   --  hardware-flow-control lines RTS (push-pull out) / CTS (in) to pads; No_Pin
   --  skips a line.  Giving Rts enables RX flow control (drive RTS, throttling
   --  the peer when our RX FIFO reaches Rx_Flow_Threshold bytes); giving Cts
   --  enables TX flow control (gate our transmitter on the peer's CTS).
   procedure Configure_Pins (B   : Bus;
                             Tx  : ESP32S3.GPIO.Optional_Pin;
                             Rx  : ESP32S3.GPIO.Optional_Pin;
                             Rts : ESP32S3.GPIO.Optional_Pin;
                             Cts : ESP32S3.GPIO.Optional_Pin;
                             Rx_Flow_Threshold : Natural);

   --  Independently re-program one frame attribute on an open port: the baud
   --  divider, or a single CONF0 field (data bits / parity / stop bits) via a
   --  read-modify-write that leaves the other fields untouched.
   procedure Set_Baud      (B : Bus; Baud   : Baud_Rate);
   procedure Set_Data_Bits (B : Bus; Bits   : Data_Bits);
   procedure Set_Parity    (B : Bus; Parity : Parity_Mode);
   procedure Set_Stop_Bits (B : Bus; Stop   : Stop_Bits);

   --  Controller-level internal TX->RX loopback (CONF0.LOOPBACK).
   procedure Set_Loopback (B : Bus; On : Boolean);

   --  Independently invert each line at the controller (CONF0 *_INV bits).
   procedure Set_Inversion (B : Bus; Tx, Rx, Rts, Cts : Boolean);

   --  Push every byte to the TX FIFO, waiting (bounded) for room.
   procedure Write (B : Bus; Data : Byte_Array);

   --  Bytes waiting in the RX FIFO.
   function Rx_Available (B : Bus) return Natural;

   --  Read up to Data'Length bytes (bounded wait per byte); Count = received.
   procedure Read (B : Bus; Data : out Byte_Array; Count : out Natural);

   procedure Close (B : in out Bus);
```

ESP32S3.Wait

Source: *src/esp32s3-wait.ads*

A tiny, profile-agnostic timeout helper: poll a condition until it holds or a deadline passes. It uses only

Ada.Real_Time (delay until) and an access-to-function, so it is lock-free and builds under EVERY profile (light-tasking, embedded, full) – handy for the common "wait for the hardware, but give up after a while" pattern without hand-rolling a deadline loop.

For a long or power-sensitive wait, prefer blocking on an interrupt (a Suspension_Object or a protected entry) over polling; see the book's "Waiting for an event" section.

Public API

```
   --  Call Ready repeatedly until it returns True or Timeout elapses; return
   --  True if the condition became true, False on timeout.  Between checks it
   --  waits Poll (default 1 ms) with `delay until`, so other tasks run.
   function Until_True
     (Ready   : access function return Boolean;
      Timeout : Ada.Real_Time.Time_Span;
      Poll    : Ada.Real_Time.Time_Span := Ada.Real_Time.Milliseconds (1))
      return Boolean;
```

ext4 filesystem (`src/ext4/`)

ESP32S3.Block_Dev

Source: `src/ext4/esp32s3-block_dev.ads`

The one block-device abstraction the filesystem talks to. A record of access-to-subprogram + an opaque context (mirrors lwext4's ext4_blockdev vtable): no tagging, no finalization, swappable at run time. Behind it sit thin adapters – ESP32S3.Block_Dev.SD_SPI_Source / SDMMC_Source on target, a file-backed device in the host test harness.

512-byte sectors (the SD logical sector). The filesystem layers its own (1 KiB .. 64 KiB) block size on top via ESP32S3.Ext4.Block_Cache.

The primitive Read/Write may RAISE Ada.IO_Exceptions.Device_Error on a hardware/IO failure (the adapters convert the SD driver's Status enum to a raise); the convenience wrappers below also raise on a null/oversize access.

Public API

```
type Sector is array (0 .. 511) of Interfaces.Unsigned_8;
type Sector_Index is new Interfaces.Unsigned_64;

type Read_Proc  is access procedure
  (Ctx : System.Address; LBA : Sector_Index; Data : out Sector);
type Write_Proc is access procedure
  (Ctx : System.Address; LBA : Sector_Index; Data : Sector);
type Count_Func is access function (Ctx : System.Address) return Sector_Index;

--  A configured backend.  Write = null marks a read-only device.
type Device is record
   Ctx   : System.Address := System.Null_Address;
   Read  : Read_Proc  := null;
   Write : Write_Proc := null;
   Count : Count_Func := null;
```

ESP32S3.Block_Dev.SD_SPI_Source

Source: `src/ext4/esp32s3-block_dev-sd_spi_source.ads`

Adapter: present an initialised SD-over-SPI card as a Block_Dev.Device for the filesystem. The Card must already be Setup + Initialize'd and must outlive the returned Device. (Embedded/full only – pulls in the finalization-based SPI stack.)

Public API

```
function Make (C : not null access ESP32S3.SD_SPI.Card) return Device;
```

ESP32S3.Block_Dev.SDMMC_Source

Source: *src/ext4/esp32s3-block_dev-sdmmc_source.ads*

Adapter: present an initialised SD card (native SDMMC host) as a Block_Dev.Device for the filesystem. The Card must already be Setup + Initialize'd and must outlive the returned Device. Unlike the SD-SPI source, SDMMC reports the card's true size (Capacity_Blocks), so Count is exact. (Embedded/full only – pulls in the finalization-based SDMMC stack.)

Public API

```
function Make (C : not null access ESP32S3.SDMMC.Card) return Device;
```

ESP32S3.Ext4

Source: *src/ext4/esp32s3-ext4.ads*

Root of the pure-Ada ext2/3/4 filesystem (a reimplementation of lwext4).

This package holds the scalar types shared by every on-disk structure, the fixed ext constants, and the exception set the whole filesystem reports through. The error model is idiomatic Ada IO: operations RAISE; the IO-family exceptions are the standard ones from Ada.IO_Exceptions (so a future Ada.Streams.Stream_IO bridge maps cleanly), plus a few filesystem-specific ones.

Targets the ESP32-S3 embedded/full runtimes (exceptions, finalization, secondary stack, heap); it also compiles host-native (x86 GNAT) for testing, since it is pure logic over the ESP32S3.Block_Dev block interface.

Public API

```
--  Unsigned scalar aliases used throughout the on-disk structures.
subtype U8  is Interfaces.Unsigned_8;
subtype U16 is Interfaces.Unsigned_16;
subtype U32 is Interfaces.Unsigned_32;
subtype U64 is Interfaces.Unsigned_64;

--  Raw byte buffers (block/sector contents, checksum inputs, names).
type Byte_Array is array (Natural range <>) of U8;

--  A filesystem block number (NOT a 512-byte sector; ext block = 1 KiB .. 64 KiB).
--  64-bit-clean for the ext4 "64bit" feature.
type Block_Number is new U64;

--  Inode numbers (ext4 inode numbers are 32-bit).
type Inode_Number is new U32;

--  Fixed inodes defined by the format.
Root_Inode    : constant Inode_Number := 2;    --  the root directory
Journal_Inode : constant Inode_Number := 8;    --  the JBD2 journal (Phase 4)
```

```
   ---------------------------------------------------------------------
   --  Error model -- exceptions.
   ---------------------------------------------------------------------

   --  Standard Ada IO exceptions, reused verbatim (same identity, so a handler
   --  for Ada.IO_Exceptions.* or for these names both catch).
   Status_Error : exception renames Ada.IO_Exceptions.Status_Error;
   Mode_Error   : exception renames Ada.IO_Exceptions.Mode_Error;
   Name_Error   : exception renames Ada.IO_Exceptions.Name_Error;
   Use_Error    : exception renames Ada.IO_Exceptions.Use_Error;
   Device_Error : exception renames Ada.IO_Exceptions.Device_Error;
   End_Error    : exception renames Ada.IO_Exceptions.End_Error;
   Data_Error   : exception renames Ada.IO_Exceptions.Data_Error;

   --  Filesystem-specific failures.
   Unsupported_Feature : exception;   --  an incompat feature we don't implement
   Bad_Checksum        : exception;   --  metadata_csum / crc mismatch
   Corrupt             : exception;   --  structural inconsistency on disk
   No_Space            : exception;   --  out of blocks or inodes
   Not_Empty           : exception;   --  rmdir on a non-empty directory
   Read_Only           : exception;   --  write attempted on a read-only mount

   ---------------------------------------------------------------------
   --  Little-endian field readers / writers over a byte buffer.  ext on-disk
   --  structures are little-endian; both x86 and the Xtensa target are too, but
   --  these keep the decoding explicit and endian-correct regardless.  Off is a
   --  0-based byte offset from B'First.
   ---------------------------------------------------------------------

   function Get_U8  (B : Byte_Array; Off : Natural) return U8;
   function Get_U16 (B : Byte_Array; Off : Natural) return U16;
   function Get_U32 (B : Byte_Array; Off : Natural) return U32;
   function Get_U64 (B : Byte_Array; Off : Natural) return U64;

   procedure Put_U8  (B : in out Byte_Array; Off : Natural; V : U8);
   procedure Put_U16 (B : in out Byte_Array; Off : Natural; V : U16);
   procedure Put_U32 (B : in out Byte_Array; Off : Natural; V : U32);
   procedure Put_U64 (B : in out Byte_Array; Off : Natural; V : U64);

   --  Big-endian variants -- the JBD2 journal stores its structures big-endian.
   function Get_U32_BE (B : Byte_Array; Off : Natural) return U32;
   procedure Put_U32_BE (B : in out Byte_Array; Off : Natural; V : U32);
```

ESP32S3.Ext4.Bitmap

Source: *src/ext4/esp32s3-ext4-bitmap.ads*

Block and inode allocation via the per-group bitmaps. Allocation sets the bit, decrements the group + superblock free counts (and bumps used-dirs for a directory inode). Write-side only – valid on a filesystem WITHOUT metadata_csum (bitmap/group-descriptor checksums are not recomputed here).

Public API

```
   --  Allocate one data block; returns its block number.  Raises No_Space.
   function Alloc_Block (V : in out Volume.Context) return Block_Number;

   --  Allocate one inode; returns its number.  As_Dir bumps the group's
   --  used-dirs count.  Raises No_Space.
   function Alloc_Inode (V : in out Volume.Context; As_Dir : Boolean)
      return Inode_Number;

   --  Release a previously-allocated block / inode (clears the bit, bumps the
   --  group + superblock free counts; Was_Dir decrements used-dirs).
   procedure Free_Block (V : in out Volume.Context; B : Block_Number);
   procedure Free_Inode (V : in out Volume.Context; N : Inode_Number;
                         Was_Dir : Boolean);
```

ESP32S3.Ext4.Block_Cache

Source: *src/ext4/esp32s3-ext4-block_cache.ads*

A small write-back LRU cache of filesystem blocks over a Block_Dev.Device.

The filesystem block size (1 KiB .. 64 KiB, a power of two and a multiple of the 512-byte sector) is fixed at Init from the superblock. Storage is heap-allocated (embedded/full have a heap); buffers may live in PSRAM since the SD backend copies to internal-SRAM scratch itself.

Copy-in / copy-out interface: callers overlay on-disk record types onto their own block buffer. Dirty blocks are written back on eviction and on Flush.

Public API

```
   type Cache is limited private;

   --  Bring up a cache of Entries blocks of Block_Size bytes over Dev.
   --  Block_Size must be a multiple of 512.
   procedure Init (C          : in out Cache;
                   Dev        : ESP32S3.Block_Dev.Device;
                   Block_Size : Positive;
                   Entries    : Positive := 32);

   --  The configured filesystem block size in bytes.
   function Block_Size (C : Cache) return Natural;

   --  Copy filesystem block B into Into (Into'Length must equal Block_Size).
   procedure Read (C : in out Cache; B : Block_Number; Into : out Byte_Array);

   --  Copy Into'Length bytes from offset Block_Off of block B into Into
   --  (Block_Off + Into'Length must be <= Block_Size).  Lets callers read just
   --  the field/entry/chunk they need without a block-sized stack buffer.
   procedure Read_At (C         : in out Cache;
                      B         : Block_Number;
                      Block_Off : Natural;
                      Into      : out Byte_Array);
```

```
   --  Replace Into'Length bytes at offset Block_Off of block B (and mark dirty).
   procedure Write_At (C         : in out Cache;
                       B         : Block_Number;
                       Block_Off : Natural;
                       From      : Byte_Array);

   --  Replace cached block B with From (length = Block_Size) and mark it dirty;
   --  written back on eviction or Flush.
   procedure Write (C : in out Cache; B : Block_Number; From : Byte_Array);

   --  Write every dirty block back to the device.
   procedure Flush (C : in out Cache);

   --  Visit the block number of every currently-dirty entry (the pending
   --  write-set, for journaling it).  The callback must not evict (it may read
   --  the dirty blocks themselves -- those are resident hits).
   procedure For_Each_Dirty
     (C     : in out Cache;
      Visit : not null access procedure (B : Block_Number));

   --  Callback-free variant of For_Each_Dirty: fill Into with the tags of the
   --  currently-dirty entries (up to Into'Length) and set Count.  Preferred on
   --  targets whose stacks are non-executable, where 'Access of a nested
   --  collector passed to For_Each_Dirty would need an unavailable trampoline.
   type Block_List is array (Positive range <>) of Block_Number;
   procedure Dirty_Tags
     (C : in out Cache; Into : out Block_List; Count : out Natural);

   --  Flush, then release all heap storage (cache unusable until re-Init).
   procedure Done (C : in out Cache);

   --  Release all storage WITHOUT flushing (discard volatile state -- used to
   --  simulate a crash in the journal tests).
   procedure Drop (C : in out Cache);
```

ESP32S3.Ext4.Block_Map

Source: *src/ext4/esp32s3-ext4-block_map.ads*

Logical-to-physical block mapping for a file's data. Dispatches on the inode's EXTENTS_FL: classic indirect block pointers (ext2/3, implemented here) or extent trees (ext4, Phase 2). A result of 0 means a sparse hole (the logical block reads as zeros).

Public API

```
   function Logical_To_Physical
     (V : in out Volume.Context; I : Inode.Info; L_Block : U64)
      return Block_Number;
```

ESP32S3.Ext4.CRC32C

Source: *src/ext4/esp32s3-ext4-crc32c.ads*

CRC32C (Castagnoli, polynomial 0x1EDC6F41, reflected) – the checksum ext4 uses for the metadata_csum feature (superblock, group descriptors, inodes, directory tails, extent blocks, bitmaps, and the journal).

`Update` is the raw building block (continue from a seed); ext4 callers seed it with feature-specific values. `Checksum` is the standard one-shot form (init ~0, final xor ~0) – self-test: Checksum ("123456789") = 16#E306_9283#.

Public API

```
   --  Continue a CRC32C over Data starting from Seed; returns the running CRC.
   function Update (Seed : U32; Data : Byte_Array) return U32;

   --  Standard CRC32C of Data (init = all-ones, final xor = all-ones).
   function Checksum (Data : Byte_Array) return U32;
```

ESP32S3.Ext4.Dir

Source: *src/ext4/esp32s3-ext4-dir.ads*

Linear directory entries (ext4_dir_entry_2): walk a directory's data blocks and read inode / rec_len / name_len / file_type / name. HTree-indexed directories still hold a valid linear layout, so this reads them too; the HTree fast-path (Phase 2) is only an optimisation.

Public API

```
   --  ext file_type values (with the FILETYPE feature).
   FT_Unknown : constant U8 := 0;
   FT_Reg     : constant U8 := 1;
   FT_Dir     : constant U8 := 2;
   FT_Symlink : constant U8 := 7;

   --  Look up Name directly in directory Dir; return its inode number, or 0 if
   --  not present.
   function Lookup (V : in out Volume.Context; Dir : Inode.Info; Name : String)
      return Inode_Number;

   --  Call Visit for every real entry (skips unused slots).  "." and ".." are
   --  reported like any other entry.
   procedure Iterate
     (V     : in out Volume.Context;
      Dir   : Inode.Info;
      Visit : not null access procedure
                (Name : String; Ino : Inode_Number; File_Type : U8));

   --  Add Name -> Child (with File_Type) to directory Dir by splitting slack in
   --  one of its existing data blocks.  Raises No_Space if no block has room
   --  (directory-block extension is a later step).
   procedure Add_Entry
     (V         : in out Volume.Context;
      Dir       : Inode.Info;
      Name      : String;
      Child     : Inode_Number;
      File_Type : U8);
```

```
   --  Remove Name from directory Dir (merging its slot into the previous entry,
   --  or zeroing its inode if first in the block).  Returns the removed entry's
   --  child inode number, or 0 if Name was not found.
   function Remove_Entry
     (V : in out Volume.Context; Dir : Inode.Info; Name : String)
      return Inode_Number;

   --  True if Dir contains only "." and ".." (i.e. is empty).
   function Is_Empty (V : in out Volume.Context; Dir : Inode.Info) return Boolean;

   --  Repoint entry Name in Dir at inode New_Ino.  Returns True if found.
   function Set_Entry_Inode
     (V : in out Volume.Context; Dir : Inode.Info; Name : String;
      New_Ino : Inode_Number) return Boolean;
```

ESP32S3.Ext4.File

Source: *src/ext4/esp32s3-ext4-file.ads*

Reading a regular file's data by byte offset, mapping logical to physical blocks through ESP32S3.Ext4.Block_Map and pulling bytes from the cache. Sparse holes read as zeros. (Write/Truncate + an RAII File handle arrive in Phase 3.)

Public API

```
   --  Read up to Into'Length bytes of file I starting at byte Offset; Last is
   --  the number actually read (0 at or past end of file).
   procedure Read
     (V      : in out Volume.Context;
      I      : Inode.Info;
      Offset : U64;
      Into   : out Byte_Array;
      Last   : out Natural);
```

ESP32S3.Ext4.FS

Source: *src/ext4/esp32s3-ext4-fs.ads*

Public facade for a mounted ext filesystem. A Mount is a limited, controlled handle: it flushes and releases the block cache automatically when it leaves scope (including on exception unwind), so there is no silently-unflushed state. Errors are raised (see ESP32S3.Ext4).

Public API

```
   type Mount is tagged limited private;

   --  Mount Dev: read + feature-gate the superblock, bring up the cache.
   --  Cache_Blocks is the number of filesystem blocks the LRU cache holds.
   procedure Open (M            : in out Mount;
                   Dev          : ESP32S3.Block_Dev.Device;
                   Read_Only    : Boolean  := True;
                   Cache_Blocks : Positive := 32);
```

```
   --  Flush + release (also done by finalization; idempotent).
   procedure Close (M : in out Mount);

   --  Resolve an absolute path to its inode number (raises Name_Error if absent).
   function Lookup (M : in out Mount; Path : String) return Inode_Number;

   --  Read inode N's metadata.
   procedure Stat (M : in out Mount; N : Inode_Number; I : out Inode.Info);

   --  Read up to Into'Length bytes of file I from byte Offset; Last = count read.
   procedure Read_File (M      : in out Mount;
                        I      : Inode.Info;
                        Offset : U64;
                        Into   : out Byte_Array;
                        Last   : out Natural);

   --  Iterate directory I's entries.
   procedure Iterate
     (M     : in out Mount;
      I     : Inode.Info;
      Visit : not null access procedure
                (Name : String; Ino : Inode_Number; File_Type : U8));

   --  Create a regular file Name in directory Dir_Path; return its inode number.
   --  (Requires a writable, non-metadata_csum volume.)
   function Create_File (M : in out Mount; Dir_Path, Name : String)
      return Inode_Number;

   --  Set the entire contents of (empty) file inode N (<= 12 * block_size bytes).
   procedure Write_File (M : in out Mount; N : Inode_Number; Data : Byte_Array);

   --  Create a subdirectory / remove a regular file / remove an empty directory.
   procedure Mkdir  (M : in out Mount; Dir_Path, Name : String);
   procedure Unlink (M : in out Mount; Dir_Path, Name : String);
   procedure Rmdir  (M : in out Mount; Dir_Path, Name : String);

   --  Rename Old_Name in Old_Dir to New_Name in New_Dir.
   procedure Rename (M : in out Mount;
                     Old_Dir, Old_Name, New_Dir, New_Name : String);

   --  Truncate file inode N to New_Size; hard-link an existing file.
   procedure Truncate (M : in out Mount; N : Inode_Number; New_Size : U64);
   procedure Link (M : in out Mount; Target_Path, New_Dir, New_Name : String);

   --  Create symbolic link Name in Dir_Path pointing at the text Target.
   procedure Symlink (M : in out Mount; Dir_Path, Name, Target : String);

   --  Atomically commit the pending write-set (the dirty metadata + the
   --  superblock, from one or more preceding operations) as a single journaled
   --  transaction: journal -> barrier -> checkpoint -> reset.  The write-set must
   --  fit the cache (so this covers metadata + small-data operations; large-file
   --  data is not journaled).  Non-csum filesystems.
   procedure Commit (M : in out Mount);
```

```
   --  (test) Commit but stop right after the barrier (simulate a crash before
   --  the checkpoint); the next mount's recovery completes it.
   procedure Commit_Crash (M : in out Mount);

   --  (test) Discard the volatile cache without checkpointing (simulate power
   --  loss); the Mount becomes inert.
   procedure Drop_Cache (M : in out Mount);

   --  The mounted volume's block size in bytes.
   function Block_Size (M : Mount) return Natural;

   --  Physical block holding logical block L_Block of file I (0 => hole).
   --  (Exposed mainly for tests.)
   function Map_Block (M : in out Mount; I : Inode.Info; L_Block : U64)
      return Block_Number;

   --  Write one committed journal transaction logging New_Data for Targets and
   --  set the RECOVER flag (does not checkpoint).  Exposed for the crash-safety
   --  test; the recovery path then applies it.
   procedure Journal_Commit (M        : in out Mount;
                             Targets  : Journal.Target_Array;
                             New_Data : Byte_Array);
```

ESP32S3.Ext4.Group_Desc

Source: *src/ext4/esp32s3-ext4-group_desc.ads*

Block-group descriptors – the per-group table that locates each group's block bitmap, inode bitmap and inode table. 32 bytes (ext2/3) or 64 bytes (the ext4 "64bit" feature). The table starts in the block right after the superblock block (First_Data_Block + 1).

Public API

```
   type Desc is record
      Block_Bitmap : Block_Number := 0;
      Inode_Bitmap : Block_Number := 0;
      Inode_Table  : Block_Number := 0;
      Free_Blocks  : U32 := 0;
      Free_Inodes  : U32 := 0;
      Used_Dirs    : U32 := 0;
```

ESP32S3.Ext4.Inode

Source: *src/ext4/esp32s3-ext4-inode.ads*

ext inode read. The 60-byte i_block area is kept raw – it is either the classic block map (12 direct + 3 indirect pointers) or, when EXTENTS_FL is set, the root of an extent tree (Phase 2); the block-map facade decides.

Public API

```
   type Info is record
      Mode       : U16 := 0;
```

```
   Size       : U64 := 0;          --  file size in bytes
   Flags      : U32 := 0;
   Links      : U16 := 0;
   Blocks_512 : U64 := 0;          --  i_blocks, in 512-byte units
   I_Block    : Byte_Array (0 .. 59) := [others => 0];  --  raw map / extent root
```

ESP32S3.Ext4.Journal

Source: src/ext4/esp32s3-ext4-journal.ads

JBD2 journal recovery (replay-on-mount). The journal lives in inode 8 as a regular file; its on-disk structures are BIG-ENDIAN (unlike the rest of ext). On a volume whose superblock has the RECOVER incompat flag set, the pending committed transactions are replayed into the filesystem before normal use, then the journal is reset and the flag cleared.

Handles the classic (non-checksummed) journal format used by ext3 and by ext4 with ^metadata_csum. A checksummed journal (CSUM_V2/V3) raises Unsupported_Feature for now.

Public API

```
-- Does the volume's superblock ask for journal recovery?
function Needs_Recovery (V : Volume.Context) return Boolean;

-- Replay committed transactions into the filesystem, honour revoke records,
-- reset the journal superblock and clear the volume's RECOVER flag.  No-op
-- if the journal is already empty.  Caller must hold a writable volume.
procedure Replay (V : in out Volume.Context);

type Target_Array is array (Positive range <>) of Block_Number;

-- Write ONE committed transaction to the journal: log New_Data (Targets'Length
-- filesystem blocks, block i held at offset (i-1)*block_size of New_Data) for
-- the given target blocks, append a commit block, point the journal at it and
-- set the fs RECOVER flag.  Does NOT checkpoint -- a crash (or the next mount's
-- Replay) applies the logged blocks to their targets.  This is the commit half
-- of the journal: a crash after Commit recovers forward, before it has no
-- effect.  Non-checksummed journals only.
procedure Commit (V        : in out Volume.Context;
                  Targets  : Target_Array;
                  New_Data : Byte_Array);

-- Commit the cache's dirty metadata blocks AND the superblock as one
-- journaled transaction: log them + write the journal commit (durable), set
-- the fs RECOVER flag (the barrier), checkpoint the metadata to its final
-- locations, then clear RECOVER and reset the journal.  A crash after the
-- barrier recovers forward (Replay); a crash before it has no effect.  This
-- is how the write path is made atomic.  Non-checksummed filesystems only.
--
-- Simulate_Crash stops right after the barrier (journal committed, RECOVER
-- set, metadata NOT yet checkpointed) -- for the crash-recovery tests.
procedure Transaction_Commit (V             : in out Volume.Context;
                              Simulate_Crash : Boolean := False);
```

ESP32S3.Ext4.Path

Source: *src/ext4/esp32s3-ext4-path.ads*

Path resolution: walk a '/'-separated absolute path from the root inode, looking each component up in its parent directory.

Public API

```
   --  Resolve Path (absolute, e.g. "/a/b/file") to its inode number.
   --  Raises Name_Error if a component is missing, Use_Error if a non-final
   --  component is not a directory.
   function Resolve (V : in out Volume.Context; Path : String)
      return Inode_Number;
```

ESP32S3.Ext4.Superblock

Source: *src/ext4/esp32s3-ext4-superblock.ads*

The ext2/3/4 superblock (1024 bytes at byte offset 1024). Read raw from the device before the block cache exists (the cache needs the block size from here). Owns the on-disk `ext4_super_block`.

Public API

```
   type Info is record
      Block_Size        : Natural := 1024;
      First_Data_Block  : U32     := 1;
      Blocks_Per_Group  : U32     := 0;
      Inodes_Per_Group  : U32     := 0;
      Inode_Size        : Natural := 128;
      Inodes_Count      : U32     := 0;
      Blocks_Count      : U64     := 0;
      Free_Blocks       : U64     := 0;
      Free_Inodes       : U32     := 0;
      Desc_Size         : Natural := 32;
      Groups_Count      : U32     := 0;
      Feature_Compat    : U32     := 0;
      Feature_Incompat  : U32     := 0;
      Feature_RO_Compat : U32     := 0;
      Has_Csum          : Boolean := False;   --  metadata_csum in effect
      Csum_Seed         : U32     := 0;        --  per-fs CRC32c seed
```

ESP32S3.Ext4.Volume

Source: *src/ext4/esp32s3-ext4-volume.ads*

The mounted-volume context shared by the operation packages (Inode, Dir, Block_Map, Path, File). Kept low in the dependency graph so those packages can `with` it without a cycle through the FS facade.

Public API

```
   type Context is limited record
      Dev       : ESP32S3.Block_Dev.Device;
```

```
   Cache     : ESP32S3.Ext4.Block_Cache.Cache;
   SB        : ESP32S3.Ext4.Superblock.Info;
   Read_Only : Boolean := True;
```

ESP32S3.Ext4.Writer

Source: *src/ext4/esp32s3-ext4-writer.ads*

High-level mutation: create a regular file and give it contents. Write support targets filesystems WITHOUT metadata_csum (ext2/3 and ext4 with ^metadata_csum); attempting it on a metadata_csum volume raises Read_Only, since the group-descriptor/bitmap/dir-tail checksums are not yet recomputed.

Public API

```
   --  Create a regular file Name in directory Dir_Path; return its inode number.
   function Create_File (V : in out Volume.Context; Dir_Path, Name : String)
      return Inode_Number;

   --  Set the entire contents of (currently empty) file inode N.  Allocates up
   --  to 12 direct blocks, i.e. up to 12 * block_size bytes (indirect-block
   --  allocation is a later step).
   procedure Write_Small (V : in out Volume.Context; N : Inode_Number;
                          Data : Byte_Array);

   --  Create an empty subdirectory Name in directory Dir_Path (with "."/"..").
   procedure Mkdir (V : in out Volume.Context; Dir_Path, Name : String);

   --  Remove regular file Name from directory Dir_Path; frees its inode + data
   --  blocks when the last link goes.  (Files with indirect/extent maps are not
   --  yet freeable -> Unsupported_Feature.)
   procedure Unlink (V : in out Volume.Context; Dir_Path, Name : String);

   --  Remove empty subdirectory Name from Dir_Path (raises Not_Empty otherwise).
   procedure Rmdir (V : in out Volume.Context; Dir_Path, Name : String);

   --  Rename Old_Name in Old_Dir to New_Name in New_Dir (same or different
   --  directory).  The target must not already exist.  Moving a directory across
   --  parents fixes up its ".." and the two parents' link counts.
   procedure Rename (V : in out Volume.Context;
                     Old_Dir, Old_Name, New_Dir, New_Name : String);

   --  Set regular file inode N's size to New_Size.  Shrinking frees the now-unused
   --  data (+ indirect) blocks; growing just extends the size (sparse).  Direct /
   --  single-indirect only.
   procedure Truncate (V : in out Volume.Context; N : Inode_Number; New_Size : U64);

   --  Create a hard link New_Name in New_Dir to the existing file Target_Path
   --  (not a directory; target must not already exist).
   procedure Link (V : in out Volume.Context;
                   Target_Path, New_Dir, New_Name : String);

   --  Create a symbolic link Name in Dir_Path whose contents is Target (the
   --  link text -- not resolved).  Short targets (< 60 bytes) are stored inline
```

```
   --  in the inode ("fast symlink"); longer ones use a single data block.  The
   --  target must not exceed one block.
   procedure Make_Symlink (V : in out Volume.Context;
                           Dir_Path, Name, Target : String);
```

Generated register layer (`svd/`)

The `svd/` directory holds 48 machine-generated packages (svd2ada from a CMSIS-SVD file; do not hand-edit). They provide the typed, bit-field register records — root `ESP32S3_Registers` — that the hand-written drivers above wrap. Regenerate with `./regenerate.sh`. The packages are:

`esp32s3_registers-aes`, `esp32s3_registers-apb_ctrl`, `esp32s3_registers-apb_saradc`, `esp32s3_registers-bb`, `esp32s3_registers-debug_assist`, `esp32s3_registers-dma`, `esp32s3_registers-ds`, `esp32s3_registers-efuse`, `esp32s3_registers-extmem`, `esp32s3_registers-gpio`, `esp32s3_registers-gpiosd`, `esp32s3_registers-hmac`, `esp32s3_registers-i2c`, `esp32s3_registers-i2s`, `esp32s3_registers-i2s1`, `esp32s3_registers-interrupt_core0`, `esp32s3_registers-interrupt_core1`, `esp32s3_registers-io_mux`, `esp32s3_registers-lcd_cam`, `esp32s3_registers-ledc`, `esp32s3_registers-pcnt`, `esp32s3_registers-peri_backup`, `esp32s3_registers-pwm`, `esp32s3_registers-rmt`, `esp32s3_registers-rng`, `esp32s3_registers-rsa`, `esp32s3_registers-rtc_cntl`, `esp32s3_registers-rtc_i2c`, `esp32s3_registers-rtc_io`, `esp32s3_registers-sdhost`, `esp32s3_registers-sens`, `esp32s3_registers-sensitive`, `esp32s3_registers-sha`, `esp32s3_registers-spi0`, `esp32s3_registers-spi1`, `esp32s3_registers-spi2`, `esp32s3_registers-system`, `esp32s3_registers-systimer`, `esp32s3_registers-timg`, `esp32s3_registers-twai`, `esp32s3_registers-uart`, `esp32s3_registers-uhci`, `esp32s3_registers-usb`, `esp32s3_registers-usb_device`, `esp32s3_registers-usb_wrap`, `esp32s3_registers-wcl`, `esp32s3_registers-xts_aes`, `esp32s3_registers`